

A Survey for LLM Agent Trajectory Analysis: From Failure Attribution to Enhancement

Junjie Wang, Yawen Wang, Mengzhuo Chen, Xiaofei Xie, Chunyang Chen,
Fangwen Mu, Zhe Liu, Qing Wang

Abstract—Large language model (LLM) agents have emerged as a transformative software paradigm, enabling autonomous task execution through natural language reasoning, multi-agent collaboration, and tool integration. However, their non-deterministic decision-making and complex interaction patterns introduce unprecedented challenges for failure diagnosis and system improvement. Unlike traditional software, where failures often manifest as code defects, agent failures are embedded in lengthy, language-heavy execution trajectories that include natural language reasoning, obscuring root causes and complicating repair. This survey provides a systematic review of the burgeoning field of trajectory analysis for failure attribution and system enhancement in LLM agents. We collect 55 papers published between early 2025 and April 2026, spanning software engineering, artificial intelligence, and human-computer interaction. We organize the literature along five key dimensions: failure taxonomy, failure attribution, system enhancement and optimization, trajectory monitoring and analysis tools, as well as datasets and benchmarks. Our analysis reveals that while the field has progressed rapidly from simple LLM prompting to causal inference, fine-tuned tracer models, and dynamic intervention, the step-level attribution accuracy remains limited, and benchmark diversity is still a bottleneck. We identify four complementary taxonomic perspectives, four methodological paradigms for attribution, and three families of enhancement strategies. We also critically assess existing benchmarks, highlighting a recent shift toward full observability and unrecoverable failure annotation. This survey serves as both a structured overview for current research and a strategic guide for advancing robust, efficient, and self-improving agent ecosystems. We further call upon the software engineering community to engage more deeply with this emerging class of software, bringing expertise in fault localization, program repair, and empirical methods to bear on the unique challenges posed by LLM agents.

Index Terms—LLM Agent, Failure Attribution, Trajectory Analysis, Agent Operations, AgentOps, Quality Assurance



1 INTRODUCTION

With the advancement of large language models (LLMs), we are witnessing a fundamental transformation in the nature of software itself. As Andrej Karpathy conceptualized in his “Software 3.0” framework [1], software is evolving from deterministic, manually coded programs (Software 1.0) and neural network weights (Software 2.0) toward a new era where natural language becomes the primary programming interface and LLMs function as the core reasoning engine. In this new paradigm, prompts effectively act as programs, and systems are built around LLM agents that can reason, plan, and execute tasks autonomously.

On the other hand, software failures are an inevitable aspect of any software product. Consequently, within software engineering research community, failure localization and repair to improve software quality are critical and highly active areas [2]–[5]. This holds equally true for LLM agents, where users frequently encounter issues such as unexpected execution breakdown and outputs that deviate from expectations [6]–[10]. Similar to traditional software, repairing these failures to restore correct system behavior

requires a non-trivial process of failure attribution, root cause identification, and subsequent fix derivation. For LLM agents, this process is even more challenging.

Unlike traditional software, where business logic is explicitly encoded in control flow and data structures, the behavior of an LLM agent is largely determined at runtime through model inference, guided by system prompts and inter-agent orchestration. Consequently, failures in such systems are not easily traceable to specific code locations; instead, they manifest in the execution trajectories, i.e., rich and unstructured logs that capture natural language reasoning, multi-step communications, and tool invocations [11], [12]. Moreover, traditional debugging techniques such as breakpoint inspection [13] or deterministic replay [14] are often inapplicable due to the language-heavy nature of LLM outputs. Thus, **trajectory analysis** becomes the primary means for failure diagnosis, attribution, and repair of LLM agents.

In classical software engineering, similar practices fall under the umbrella of log analysis, which has long been employed for system diagnostics [15]–[18]. However, the language-heavy, reasoning-driven, and context-dependent nature of agent trajectories introduces new complexities that challenge traditional log analysis techniques.

First, the inadequacy of structured log analysis for language-heavy, reasoningdriven trajectories. Traditional logs are structured and deterministic, enabling techniques like parsing and anomaly detection [18], [19]. Agent trajectories, by contrast, are typically recorded in structured

- J. Wang, Y. Wang, M. Chen, F. Mu, Z. Liu, Q. Wang are with State Key Laboratory of Intelligent Game, Institute of Software Chinese Academy of Sciences, and University of Chinese Academy of Sciences, Beijing, China. J. Wang and Q. Wang are corresponding authors.
E-mail: {junjie, yawen2018, liuzhe2020, wq}@iscas.ac.cn
- X. Xie is with Singapore Management University, Singapore.
E-mail: xfxie@smu.edu.sg
- C. Chen is with Technical University of Munich, Munich, Germany
E-mail: chunyang.chen@monash.edu

formats, but their key contents, such as intermediate reasoning, inter-agent communication, and tool-related messages, are largely expressed in natural language, vary across runs, and may include hallucinated or redundant steps. This makes the techniques developed for conventional logs substantially more brittle in agent settings.

Second, the inability of external observation to capture internal reasoning. Traditional analysis infers faults from external events (e.g., error codes, performance metrics) [20]. Agent failures originate in internal decision processes, requiring techniques that can reconstruct why an agent acted as it did a capability beyond traditional logs.

Third, the mismatch between code-level fixes and system-level repairs. Fault localization in traditional software pinpoints code lines for patching [21]. Agent failures can stem from prompts, workflows, or configurations, where fixes are system-wide. Ineffective localization techniques would offer little guidance for such high-level remediation.

Fourth, the gap between ensuring stability and optimizing capability. Log analysis traditionally focuses on correctness and efficiency. For agents operating in open-ended environments, the goal extends to improving performance in unforeseen scenarios, a shift from ensuring system stability to capability assessment that current methods cannot fully support.

Given that established techniques for failure localization, repair, and log analysis in traditional software can not be directly applicable to LLM agents, a new stream of research has emerged in recent years. This body of work focuses on failure attribution and repair for LLM agents, leveraging trajectory analysis to enable deep understanding and systematic optimization. In this work, we present a systematic survey of this burgeoning field, i.e., trajectory analysis for failure attribution and system enhancement in LLM agents.

Research in this direction only began to emerge in 2025, yet within the short span of 16 months from early 2025 to the present, we have already collected 55 relevant papers. This rapid growth indicates the field has garnered significant attention. These papers are published across conferences in software engineering, artificial intelligence, human-computer interaction, and others, reflecting broad, interdisciplinary interest in the challenges of LLM agent failure analysis. Among these collected studies, **failure attribution** and **system enhancement and optimization** constitute the core technical pursuits for diagnosing and improving LLM agents. The other three, **failure taxonomy**, related **datasets and benchmarks**, **trajectory monitoring and analysis tools** provide essential support. As demonstrated in Figure 1, we identify four complementary taxonomic perspectives, four methodological paradigms for attribution, and three families of enhancement strategies. Our analysis also reveals that while the field has progressed rapidly from simple LLM prompting to causal inference, fine-tuned tracer models, and dynamic intervention, the step-level attribution accuracy remains limited, and benchmark diversity is still a bottleneck. Beyond summarizing existing work, this survey identifies critical gaps and future directions.

This survey thus serves not only as a structured overview of current research but also as a strategic guide for advancing the reliability and efficiency of LLM agents. This survey also complements the emerging AgentOps [67] discipline by focusing on its trajectory-centered diagnostic and improvement foundations. While much of the existing work has originated in the AI community, we hope this survey serves as a clear call for increased engagement from software engineering researchers, who bring deep expertise in fault localization, program repair, and empirical methods to bear on this emerging class of software. Furthermore, by grounding diagnosis in trajectory analysis and linking it to systemic improvement, this field moves closer to realizing robust, self-improving agent ecosystems capable of operating reliably in real-world environments.

2 BACKGROUND

2.1 Execution and Trajectory of LLM Agents

We consider an LLM agent $\mathcal{M}$ composed of a finite set of agents $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$ that collaboratively perform a task τ . The system executes in discrete steps. At each step, a single agent is selected to act, possibly through a dispatcher, router, or planning component that is part of the system. At step t , the acting agent is selected as $a(t) = g(h_{t-1})$, where $g(\cdot)$ represents the system-level agent selection mechanism and h_{t-1} denotes the system context at step $t-1$, summarizing the execution history. The selected agent receives an input $x_t = \phi(h_{t-1})$, and produces an output $y_t = \pi_{a(t)}(x_t)$, where $\pi_{a(t)}$ denotes the policy of agent $a(t)$. The system context is then updated as $h_t = u(h_{t-1}, a(t), x_t, y_t)$. The execution terminates after T steps, producing an execution trace $\mathcal{T} = \langle s_1, s_2, \dots, s_T \rangle$. Each execution step s_t is associated with a structured observable record $o_t = (\text{step_id}_t, \text{agent_id}_t, a(t), x_t, y_t, \dots)$.

We present part of an example execution trajectory below; for a more comprehensive illustration, readers may refer to benchmark studies like [11], [46].

```
...
"step_id": 1,
"agent_id": 0,
"agent_name": "PlannerAgent",
"input_context": "Task: Plan a three-day Walt
Disney World visit for a family.",
"output_content": "I will break the task into
park selection, ticket options, and visit
order."

"step_id": 2,
"agent_id": 1,
"agent_name": "SearchAgent",
"input_context": "Find the main theme parks at
Walt Disney World.",
"output_content": "Searching official Disney
sources.",
"tool_logs": {
  "tool_name": "web_search",
  "input_arguments": "Walt Disney World theme
parks",
  "outputs": "Magic Kingdom, EPCOT, Hollywood
Studios, and Animal Kingdom.",
  "execution_status": "success"
}
...
```

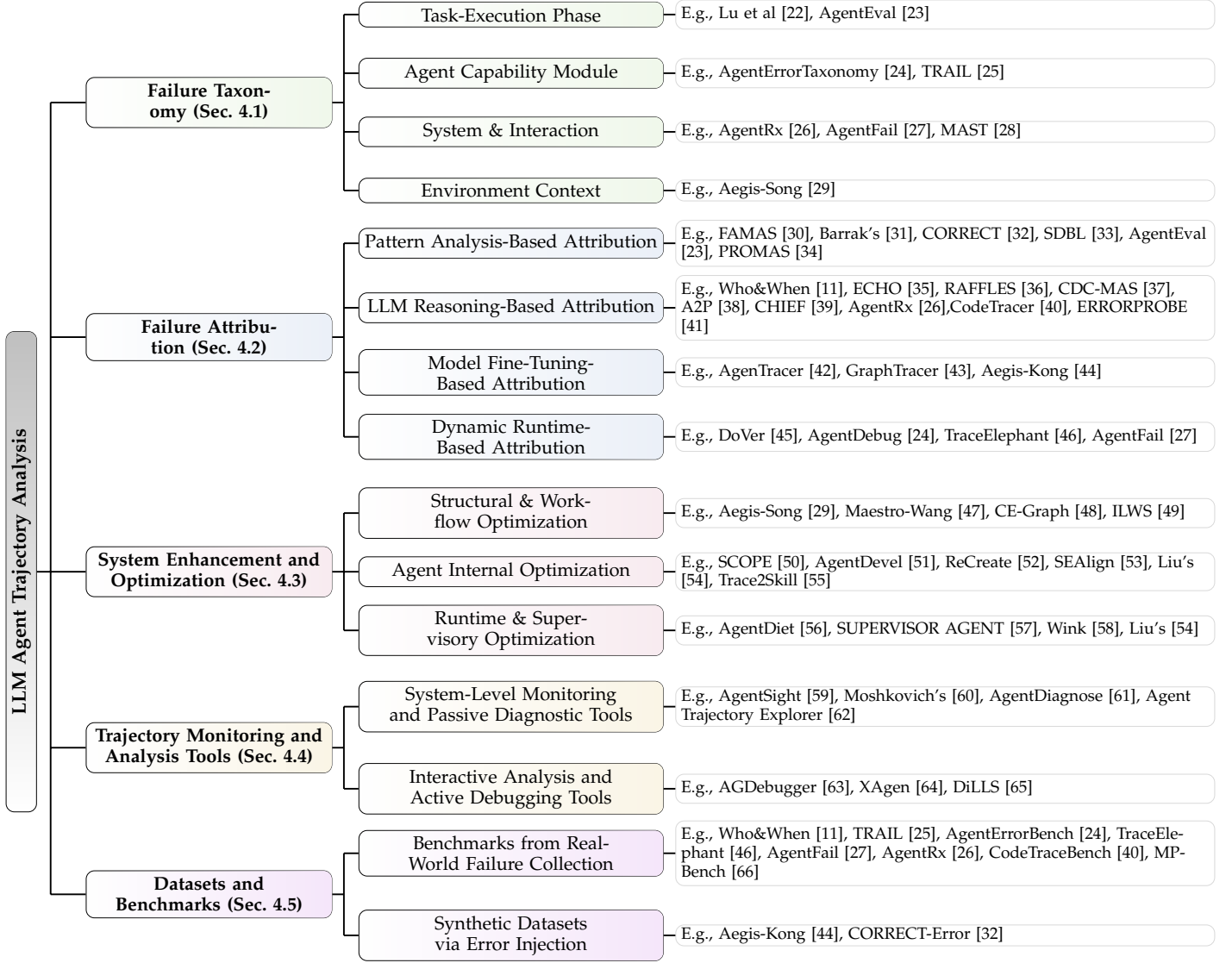


Fig. 1: Overview of related studies about trajectory analysis of LLM agents.

2.2 Failure Attribution and Enhancement

Given a completed execution trace $\mathcal{T}$, the system produces a task outcome. We define a task outcome function $\Omega(\mathcal{T}) \in \{0,1\}$, where $\Omega(\mathcal{T}) = 1$ indicates successful task completion and $\Omega(\mathcal{T}) = 0$ indicates failure. Failure attribution aims to identify the agent and the execution step responsible for a task failure based on observable execution traces. We define failure attribution at two levels. **Step-level attribution.** Let $\mathcal{T}_{\leq t}$ denote the prefix of the execution trace up to step t . We define the set of feasible continuations from $\mathcal{T}_{\leq t}$ under system $\mathcal{M}$ as $\mathcal{C}(\mathcal{T}_{\leq t}) = \{\mathcal{T}' \mid \mathcal{T}' \text{ is a valid continuation from } \mathcal{T}_{\leq t}\}$. Failure is said to become inevitable at step t if all feasible continuations result in failure: $\forall \mathcal{T}' \in \mathcal{C}(\mathcal{T}_{\leq t}), \Omega(\mathcal{T}_{\leq t} \oplus \mathcal{T}') = 0$. The failure step is defined as the earliest inevitable step $t^* = \min\{t \in \{1, \dots, T\} \mid \forall \mathcal{T}' \in \mathcal{C}(\mathcal{T}_{\leq t}), \Omega(\mathcal{T}_{\leq t} \oplus \mathcal{T}') = 0\}$. **Agent-level attribution.** Given an execution trace $\mathcal{T}$ and its failure step t^* , agent-level attribution identifies the agent whose action at the failure step gives rise to the task failure. Formally, the failure-responsible agent is defined

as $a^* = \arg \max_{a \in \mathcal{A}} \mathbb{I}[\exists s_{t^*} \in \mathcal{T} \text{ s.t. } a(t^*) = a]$, where $a(t^*)$ denotes the agent selected by the system to act at step t^* in the execution trace $\mathcal{T}$.

Given an agent $\mathcal{M}$ and a performance objective $\mathcal{J}(\mathcal{M})$ that quantifies its overall capability, such as task success rate, execution efficiency, or robustness in uncertain environments, system enhancement and optimization refers to the process of systematically transforming $\mathcal{M}$ into an improved system $\mathcal{M}'$ by modifying its constituent components (e.g., the agent policies π_a , the selection mechanism $g(\cdot)$, the context-update function $u(\cdot)$, or the input-formation function $\phi(\cdot)$). The goal is to maximize the objective $\mathcal{J}(\mathcal{M}')$ subject to constraints on allowable modifications, often guided by insights derived from the analysis of execution traces $\mathcal{T}_1, \mathcal{T}_2, \dots$ produced by the original system.

Note that, the above formalization is intended mainly as a conceptual scaffold for this survey rather than a comprehensive model of all LLM agents. It adopts a simplified sequential view of execution because this matches the dominant form of systems studied in the current literature.

Moreover, our notion of the failure step is role-aware and recoverability-aware, rather than purely based on chronological counterfactual precedence, following common practice [26], [46]. In multi-agent workflows, earlier mistakes may remain recoverable if later verifier or supervisory components are explicitly responsible for detecting and correcting them. We therefore attribute failure to the point where a designated recovery opportunity was still available but not taken, i.e., where failure became effectively inevitable within the systems architecture.

2.3 Demonstration of LLM Agent Trajectory Analysis

Figure 2 demonstrates an example LLM Agent trajectory, related failure attribution and enhancement.

The task is to split the last three Amazon orders among the user and three roommates, then create one Venmo payment request per roommate. The failure occurs at *step S4*, where the agent attempts to call the Venmo API to create payment requests. The root cause is that the agent omits the required `user_email` argument when invoking `venmo.create_payment_request`, causing an exception that is silently swallowed. To fix this, an enhancement prompt is added: before calling any state-changing API, the agent must inspect the exact API documentation, read the documented parameter list, explicitly identify every required parameter, and provide all of them explicitly. This ensures that the required `user_email` argument is no longer missed.

Note that this is a highly simplified example intended only to help readers intuitively understand the concept; real-world scenarios are considerably more complex.

3 PAPER SELECTION AND REVIEW SCHEMA

3.1 Paper Collection Methodology

To ensure comprehensive coverage of relevant literature, we employed both automated search (across academic databases) and manual search (within major software engineering and artificial intelligence venues). Our search period spans from January 2023 to February 2026, with a supplementary search conducted for recent publications from March 2026 to April 2026. The paper collection process is demonstrated in Figure 3.

3.1.1 Automated and Manual Search

To capture research from diverse subfields, we performed an extensive search across four major scientific databases: ACM Digital Library, IEEE Xplore, arXiv, and DBLP.

We initially searched for papers whose titles contained keywords related to LLM agents and trajectory analysis (listed below) in the first three databases. For DBLP, we added keywords related to failure analysis and debugging to filter out irrelevant studies, as relying solely on agent-related keywords would yield an excessively large candidate set. While using two keyword sets for DBLP may omit some studies, we consider this a feasible strategy since many relevant papers from this database are also indexed in the other three databases.

- **Keywords related to both LLM agents and trajectory analysis:** *agent trajectory* OR *LLM agent* OR *multi-agent*

system OR *agent failure* OR *agent trace* OR *agent debugging* OR *agent monitoring* OR *agent interaction* OR *agent coordination* OR *agent workflow*.

- **Keywords related to failure analysis and debugging:** *failure attribution* OR *error localization* OR *root cause analysis* OR *debugging* OR *fault diagnosis* OR *trajectory visualization* OR *agent observability* OR *system enhancement*.

We further filtered papers based on their content, retaining only those that mention “LLM”, “language model”, “agent trajectory”, or specific LLM names. This step helps eliminate studies not involving neural or generative agents.

To complement automated searches and reduce omission risk, we conducted a manual search within the proceedings of top-tier software engineering conferences and journals, as well as the major artificial intelligence venues considering interdisciplinary nature of LLM agent research.

More than that, after completing the searches and quality assessment, we performed backward snowballing by reviewing the references of collected papers.

3.1.2 Inclusion and Exclusion Criteria

Our search strategy was intentionally broad to maximize initial coverage. To ensure relevance, we defined the following inclusion and exclusion criteria, applied manually to each paper.

Inclusion Criteria:

- The paper proposes or evaluates methods for failure attribution, trajectory analysis, or debugging of LLM agents.
- The paper introduces datasets, benchmarks, or tools for analyzing or visualizing agent trajectories.
- The paper presents empirical studies or frameworks for enhancing or optimizing LLM agents via trajectory analysis.
- The paper focuses on multi-agent coordination, interaction protocols, or system-level observability in LLM agent contexts.

Exclusion Criteria:

- The paper does not involve LLM agents (e.g., focuses solely on traditional software or single-model applications)¹.
- The paper uses non-generative or encoder-only models (e.g., BERT) without agentic reasoning or interaction.
- The paper only mentions LLM agents tangentially (e.g., in future work or background sections).
- The paper focuses on testing LLM performance (e.g., fairness, security) rather than agent behavior.
- The paper is a tool evaluation without proposing analytical or diagnostic methods for agent trajectories.

Two authors independently reviewed each paper against these criteria. Disagreements were resolved by a third author.

1. This survey focuses on “LLM agents” in which the model participates in multi-step task execution with at least some combination of the following properties: 1) iterative reasoning or planning; 2) interaction with tools, environment, or external memory; 3) multi-step state evolution across a trajectory; 4) collaboration/orchestration among multiple components or agents. By contrast, we exclude “single-turn or purely generative single-model applications” where the LLM produces an answer without trajectory-like execution or agentic interaction.

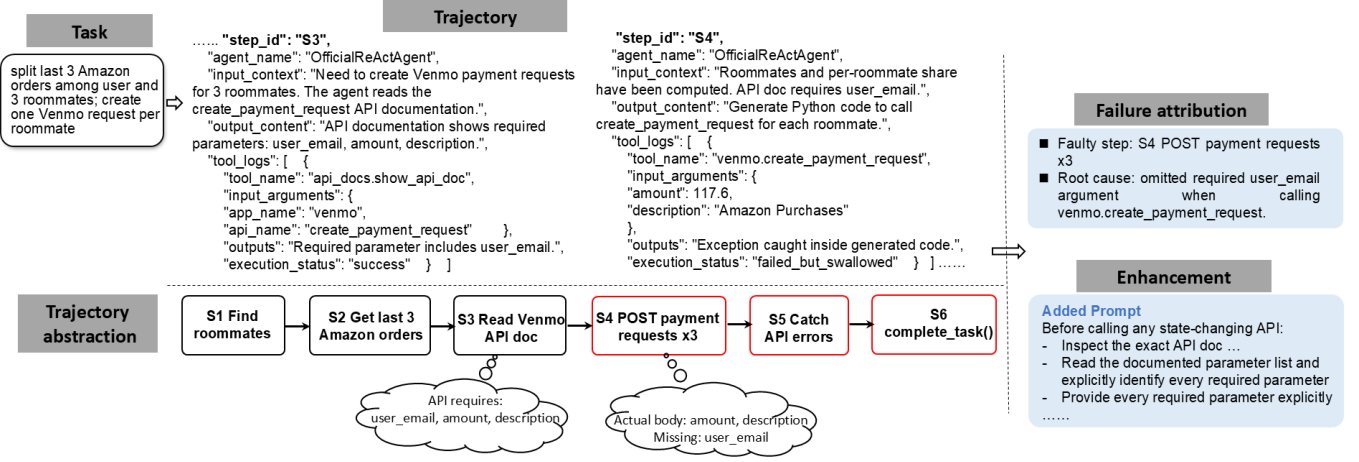


Fig. 2: Demonstration about LLM agent trajectory analysis

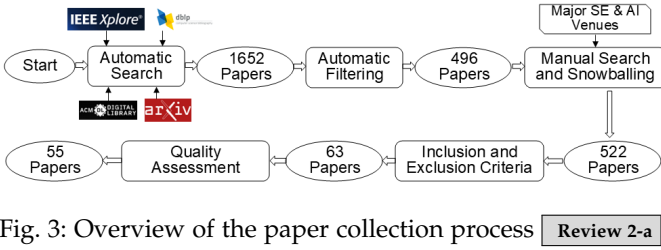


Fig. 3: Overview of the paper collection process

3.1.3 Quality Assessment

We further applied a quality assessment checklist to exclude low-quality studies. Specifically, two authors independently assessed the quality of the candidate papers using the 10-item checklist. They have research experience in software engineering, LLM agents, and empirical literature analysis. For each paper, the annotators first scored the items independently as yes (1), partial (0.5), or no (0). They then compared their assessments and discussed discrepancies. If consensus could not be reached directly, the disagreement was resolved through discussion with an additional author acting as an adjudicator.

- Is the research goal clearly stated and relevant to LLM agent trajectory analysis or failure diagnosis?
- Is the proposed technique or framework clearly defined and repeatable?
- Does the paper make a clear contribution to the understanding or improvement of LLM agents?
- Are the LLM models used explicitly described?
- Is the method of applying LLMs for analysis or attribution explained?
- Is there a clear validation methodology (e.g., datasets, benchmarks, case studies)?
- Are the selected evaluation subjects appropriate for the research goals?
- Are baselines or control methods included to demonstrate effectiveness?
- Are the evaluation metrics relevant to the stated objectives?

- Are the results clearly presented and aligned with the research goals?

If a question is not applicable to a given paper, we exclude it from that paper's assessment, and normalize the final quality score over the remaining applicable questions. This avoids unfairly penalizing papers whose methodological design does not involve certain aspects. For example, if a paper has two questions deemed not applicable, the maximum achievable score becomes 8 instead of 10. The final quality score is then normalized to a 10-point scale (e.g., an actual score of 8 out of 8 would be recorded as 10 out of 10), ensuring fair comparison across papers with different sets of applicable criteria. Finally, papers with a total score below 8 were excluded.

3.2 Collection Results

Our initial search retrieved 1,652 papers. After automated filtering, manual screening, application of inclusion/exclusion criteria, and quality assessment, we retained 55 papers relevant to LLM agent trajectory analysis for failure attribution and system enhancement. This significant reduction stems from our deliberately broad initial keyword design, i.e., intended to capture interdisciplinary work, followed by rigorous filtering that removed papers about tangential mentions of agents without substantive contribution to trajectory analysis. Table 1 provides a detailed breakdown of the collected papers.

3.3 General Overview of Collected Papers

Figure 4 illustrates the publication timeline distribution. It can be observed that although LLM agents emerged around 2023, observable research outcomes on trajectory analysis began to appear only in 2025 and rapidly grew into a substantial body of work within that same year. Note that, this does not mean that there was no related work before 2025. Rather, earlier work tended to fall into adjacent but distinct categories, such as: general LLM observability, agent framework design, multi-agent coordination. None of them focus

TABLE 1: Overview of collected studies

ID	Dimension	Paper Title	Approach Name	Date	Venue	Ref.
1	Taxonomy; Benchmark	TRAIL: Trace Reasoning and Agentic Issue Localization	TRAIL	202506	arXiv	[25]
2	Taxonomy; Enhancement	Aegis: Taxonomy and Optimizations for Overcoming Agent-environment Failures in LLM Agents	Aegis-Song	202508	arXiv	[29]
3	Taxonomy	Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks	Lu’s	202508	ASE	[22]
4	Taxonomy; Attribution; Benchmark	Where LLM Agents Fail And How They Can Learn From Failures	AgentDebug, AgentError-Taxonomy, AgentErrorBench	202509	arXiv	[24]
5	Taxonomy	Why Do Multiagent Systems Fail	MAST	202510	Workshop	[28]
6	Taxonomy	How Do LLMs Fail In Agentic Scenarios	Roig’s	202512	NeurIPS	[68]
7	Taxonomy; Attribution; Benchmark	AgentRx: Diagnosing AI Agent Failures from Execution Trajectories	AgentRx	202602	arXiv	[26]
8	Taxonomy; Attribution; Benchmark	Demystifying the Lifecycle of Failures in Platform-Orchestrated Agent Workflows	AgentFail	202602	arXiv	[27]
9#	Taxonomy; Attribution	AgentEval DAG-Structured Step-Level Evaluation	AgentEval	202604	arXiv	[23]
10	Attribution; Benchmark	Which Agent Causes Task Failures and When: On Automated Failure Attribution of LLM Multi-Agent Systems	Who&When	202506	ICML	[11]
11	Attribution	RAFFLES: Reasoning-based Attribution of Faults for LLM Systems	RAFFLES	202508	Workshop	[36]
12	Attribution	Automatic Failure Attribution and Critical Step Prediction Method for Multi-Agent Systems Based on Causal Inference	CDC-MAS	202508	arXiv	[37]
13	Attribution	Who is Introducing the Failure? Automatically Attributing Failures of Multi-Agent Systems via Spectrum Analysis	FAMAS	202508	FSE	[30]
14	Attribution; Benchmark	CORRECT: Condensed Error Recognition via Knowledge Transfer in Multi-Agent Systems	CORRECT	202508	arXiv	[32]
15	Attribution	Abduct, Act, Predict: Scaffolding Causal Inference for Automated Failure Attribution in Multi-Agent Systems	A2P	202509	NeurIPS	[38]
16	Attribution	AgentTracer: Who Is Inducing Failure in the LLM Agentic Systems	AgentTracer	202509	ICLR	[42]
17	Attribution	Where Did It All Go Wrong: A Hierarchical Look into Multi-Agent Error Attribution	ECHO	202510	NeurIPS	[35]
18	Attribution	GraphTracer: Graph-Guided Failure Tracing in LLM Agents for Robust Multi-turn Deep Search	GraphTracer	202510	arXiv	[43]
19	Attribution; Benchmark	Aegis: Automated Error Generation and Attribution for Multi-Agent Systems	Aegis-Kong	202510	arXiv	[44]
20	Attribution	Traceability and Accountability in Role-Specialized Multi-Agent LLM Pipelines	Barrak’s	202510	arXiv	[31]
21	Attribution	DoVer: Intervention-Driven Auto Debugging for LLM Multi-Agent Systems	DoVer	202512	ICLR	[45]
22	Attribution	From Flat Logs to Causal Graphs Hierarchical Failure Attribution for LLM-based Multi-agent Systems	CHIEF	202601	arXiv	[39]
23	Attribution	Scope Delineation Before Localization A Two-Stage Framework for Enhancing Failure Attribution in Multi-Agent Systems	SDBL	202601	AAAI	[33]
24	Attribution; Benchmark	Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems	TraceElephant	202601	ACL	[46]
25#	Attribution	ProMAS: Proactive Error Forecasting for Multi-Agent Systems Using Markov Transition Dynamics	PROMAS	202603	arXiv	[34]
26#	Attribution	Towards Self-Improving Error Diagnosis in Multi-Agent Systems	ERRORPROBE	202604	arXiv	[41]
27#	Attribution; Benchmark	CodeTracer: Towards Traceable Agent States for Failure Attribution	CodeTracer	202604	arXiv	[40]
28	Enhancement	Maestro: Joint Graph Config Optimization for Reliable AI Agents	Maestro-Wang	202509	arXiv	[47]
29	Enhancement	Improving the Efficiency of LLM Agent Systems through Trajectory Reduction	AgentDiet	202509	arXiv	[56]
30	Enhancement	Failure-Driven Workflow Refinement	CE-Graph	202510	arXiv	[48]
31	Enhancement	Stop Wasting Your Tokens: Towards Efficient Runtime Multi-Agent Systems	SUPERVISOR AGENT	202510	FSE	[57]
32	Enhancement	Instruction-Level Weight Shaping: A Framework for Self-Improving AI Agents	ILWS	202512	arXiv	[49]
33	Enhancement	SCOPE: Prompt Evolution for Enhancing Agent Effectiveness	SCOPE	202512	arXiv	[50]
34	Enhancement	AgentDevel: Reframing Self-Evolving LLM Agents as Release Engineering	AgentDevel	202601	arXiv	[51]
35	Enhancement	ReCreate: Reasoning and Creating Domain Agents Driven by Experience	ReCreate	202601	ACL	[52]
36#	Enhancement	Wink: Recovering from Misbehaviors in Coding Agents	Wink	202602	arXiv	[58]
37#	Enhancement	Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills	Trace2Skill	202603	arXiv	[55]
38#	Enhancement	Trajectory-Informed Memory Generation for Self-Improving	Fang’s	202603	arXiv	[69]
39#	Enhancement	SEAlign: Alignment Training for Software Engineering Agent	SEAlign	202604*	ICSE	[53]
40#	Enhancement	Process-Centric Analysis of Agentic Software Systems	Liu’s	202604	OOPSLA	[54]
41	Monitoring	Agent Trajectory Explorer: Visualizing and Providing Feedback on Agent Trajectories	Agent Trajectory Explorer	202501*	AAAI	[62]
42	Monitoring	Interactive Debugging and Steering of Multi-Agent AI Systems	AGDebugger	202504*	CHI	[63]
43	Monitoring	AgentSight: System-Level Observability for AI Agents using eBPF	AgentSight	202508	Workshop	[59]
44	Monitoring	Taming Uncertainty via Automation: Observing, Analyzing, and Optimizing Agentic AI Systems	Moshkovich’s	202511	arXiv	[60]
45	Monitoring	AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories	AgentDiagnose	202511*	EMNLP	[61]
46	Monitoring	XAgent: An Explainability Tool for Identifying and Correcting Failures in Multi-Agent Workflows	XAgent	202512	CHI	[64]
47	Monitoring	DiLLS: Interactive Diagnosis of LLM-based Multi-agent Systems via Layered Summary of Agent Behaviors	DiLLS	202602	CHI	[65]
48#	Benchmark	Rethinking Failure Attribution in Multi-Agent Systems: A Multi-perspective Benchmark and Evaluation	MP-Bench	202603	arXiv	[66]
49	Others	Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories	Bouzenia’s	202510	ASE	[12]
50	Others	MAESTRO: Multi-Agent Evaluation Suite for Testing Reliability and Observability	MAESTRO-Ma	202601	arXiv	[70]
51	Others	Trajectory Guard A Lightweight, Sequence-Aware Model for Real-Time Anomaly Detection in Agentic AI	Trajectory Guard	202601	Workshop	[71]
52#	Others	From Features to Actions: Explainability in Traditional and Agentic AI Systems	Chaduvula’s	202602	arXiv	[72]
53#	Others	Signals: Trajectory Sampling and Triage for Agentic Interactions	Chen’s	202604	arXiv	[73]
54#	Others	Beyond Final Code: A Process-Oriented Error Analysis of Software Development Agents in Real-World GitHub Scenarios	Chen’s	202604	arXiv	[74]
55	Others	Beyond Resolution Rates: Behavioral Drivers of Coding Agent Success and Failure	Mehtiyev’s	202604	arXiv	[75]

Note: In the Date column, entries marked with an asterisk (*) indicate the publication date of the paper. For all other entries, the date represents when the paper was posted on arXiv, which is typically earlier than the actual publication date. Paper 2 and 19, Paper 30 and 50 have the same method name, and we distinguish them with the surname of first author, e.g., Aegis-Song.

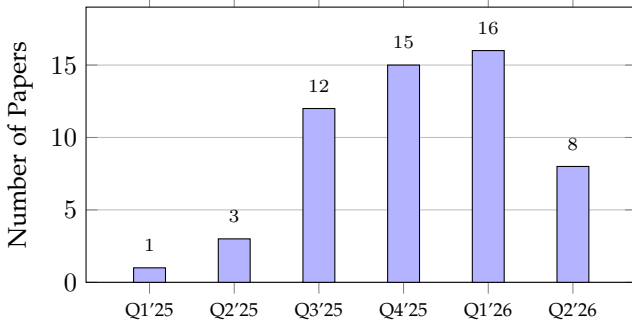


Fig. 4: Quarterly distribution of papers from Q1 2025 to Q2 2026 (Q2 2026 includes only Apr.)

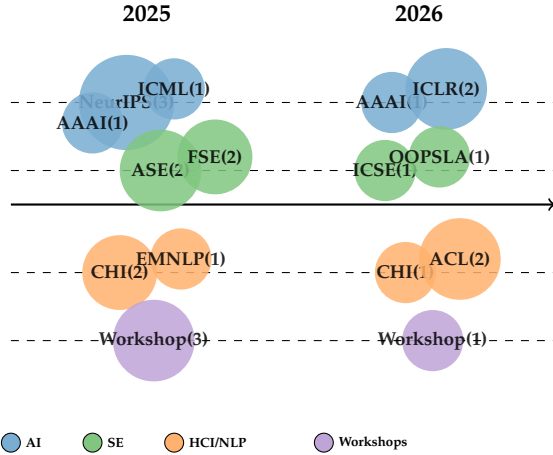


Fig. 5: Timeline and venue distribution of collected studies (arXiv papers excluded)

on trajectory-based failure attribution or enhancement. Furthermore, with 24 papers already published in just the first four months of 2026, this field continues to develop rapidly.

As demonstrated in Figure 5, while failure attribution and repair are popular topics in software engineering, papers from software engineering venues currently account for only 11% (6/55) of all publications, lower than the 15% (8/55) from AI venues. Moreover, one of the most influential early works (Who&When) originated from the AI community (ICML 2025 Spotlight). This highlights the interdisciplinary nature of the task, with researchers from multiple fields actively contributing to the topic.

Among the collected publications, while 56% (31/55) currently exist as arXiv preprints. Given the rapid evolution of this field, many significant contributions first appear as preprints before formal publication; we therefore include them after applying our quality assessment checklist (Section 3.1.3) to ensure relevance and technical soundness. This aligns with common practice in survey papers covering emerging topics, where capturing the latest advances often requires incorporating preprint research. While these preprints have not undergone formal peer review, they represent active and often influential contributions to the ongoing discourse.

4 LLM AGENT TRAJECTORY ANALYSIS

We group the related studies in this field along five key dimensions, as illustrated in Figure 6. The research on quality assurance about LLM agents is organized around two central goals: finding out why failures happen and using that knowledge to build better systems. In our view, the core challenges of the field are **failure attribution** (pinpointing the location and root cause of the failure within complex interactions) and **system enhancement and optimization** (applying those insights to fix and improve agents). To support this core, three areas provide the necessary foundation. First, **failure taxonomies** offer a common schema to classify failures, which is essential for analyzing and discussing failures systematically. Second, **trajectory monitoring and analysis tools** supply tools needed to observe agent behavior, enabling both automated analysis and human-in-the-loop debugging. Finally, specialized **datasets and benchmarks** are vital for fairly evaluating and comparing different attribution and enhancement methods, ensuring progress is measurable and solid.

Figure 7 illustrates the distribution of research dimensions among the 55 surveyed studies. The data reveals a clear concentration of effort on **failure attribution**, which appears in 22 papers, underscoring its role as the central technical challenge in diagnosing LLM agents. Following that, **enhancement** accounts for 14 papers, while **benchmark** and **taxonomy** each account for 10 and 9 papers, reflecting a balanced focus on both understanding failure modes and improving system performance. There are 7 papers mainly related with the empirical studies about trajectories of such systems, and we put them into others.

We organize the analysis into subsections, with each subsection focusing on one specific dimension.

4.1 Failure Taxonomy

As mentioned earlier, as an emerging form of software, LLM agents exhibit failures whose manifestations and characteristics are substantially different from those of traditional software. Consequently, several researchers have sought to analyze why or how these systems fail. They examine the trajectories generated by these systems to diagnose the root causes of failures. Moving beyond individual failure instances, they also develop a structured understanding of why or how failures occur. This effort has led to the creation of many failure taxonomies for such systems.

4.1.1 Categorizing Taxonomies

Researchers have proposed a variety of taxonomies to structure the understanding of failures in LLM agents. These efforts, while diverse in their specific categorizations, can be synthesized by examining the views through which they analyze system failures. To provide a more streamlined comparative perspective, we organize existing work around four guiding questions: (1) *where* in the task pipeline did the failure occur? (2) *what* core capability of the agent failed? (3) *who* (which agent or system component) is responsible? and (4) *why* did the failure arise under the given environment context?

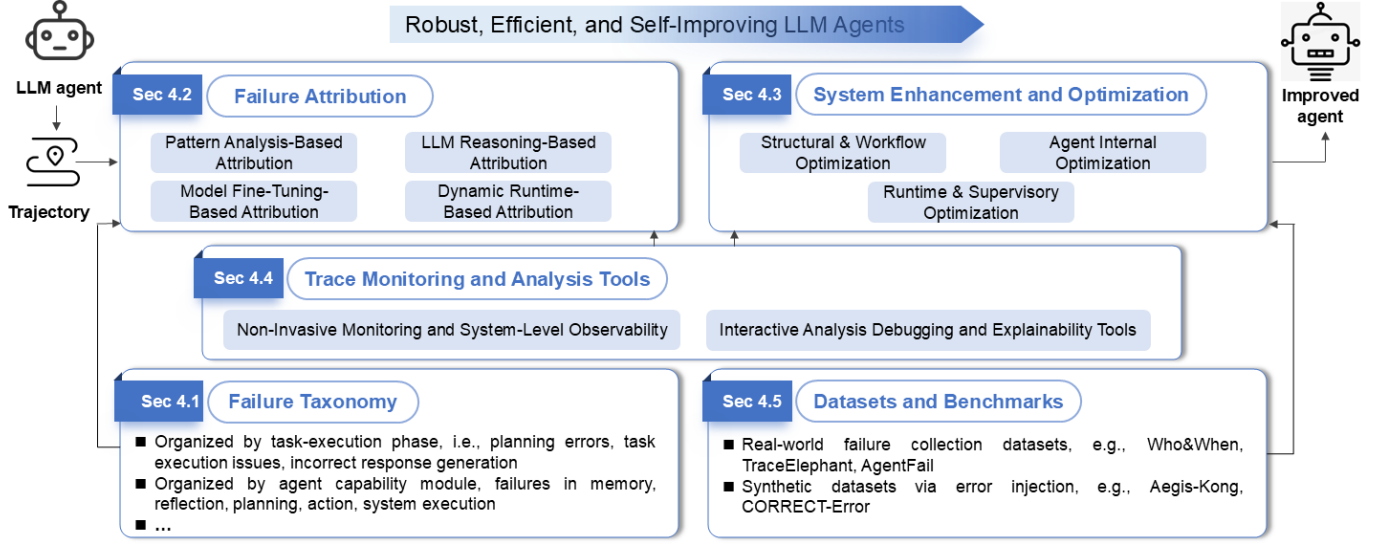


Fig. 6: Overview of five key dimensions in trajectory analysis of LLM agents

TABLE 2: Overview of failure taxonomy studies

Taxonomy Perspective	Name	Lv.	L1	L1 Category	Total	Top 3 Failures	Dist.	Improv.	Type	Source Systems (Tasks)
Task-Execution Phase	Lu et al. [22]	3	3	Task planning, task execution, response generation	14	Improper task decomposition, incorrect code, max round limits	Partial	Partial	MAS	TaskWeaver, MetaGPT, AutoGen
	AgentEval [23]	3	3	Planning, execution, integration	21	Parameter errors, wrong tool selection, goal misinterpretation	Yes	No	Both	N/A (SWE-bench, τ -bench)
Agent Capability Module	Agent Error Taxonomy [24]	2	5	Memory, reflection, planning, action, system	17	Inefficient plan, over simplification, progress misjudge	Yes	No	SAS	N/A (ALFWorld, GAIA, WebShop)
	TRAIL [25]	3	3	Reasoning, system execution, planning/coordination	20	Formatting errors, instruction non-compliance, goal deviation	Yes	No	Both	OpenDeepResearch (GAIA), CodeAct (SWE-Bench Lite)
System & Interaction	AgentRx [26]	2	3	Design, coordination, verification	9	Misinterpretation of tool output, instruction adherence, under-specified intent	Yes	No	Both	τ -bench, Flash, Magentic-One
	AgentFail [27]	2	3	Agent, workflow, platform	16	Knowledge / reasoning limitation, poor prompt design, response deviation	Yes	Yes	MAS	10 systems from Dify/Coze (HumanEval, SWE-Bench, TravelPlanner)
	MAST [28]	2	3	Specification issues, inter-agent misalignment, task verification	14	Step repetition, reasoning-action mismatch, fail to ask clarification	Yes	Partial	MAS	MetaGPT, ChatDev, HyperAgent, AppWorld, AG2
Environment Context	Aegis-Song [29]	2	3	Exploration, exploitation, resource exhaustion	6	State-space navigation failure, instruction following failure, tool output processing failure	Yes	Yes	SAS	5 systems (AgentOccam, CRMArena, etc)

Note: Lv., L1, Total, Dist. Improv. are respectively short for taxonomy levels (Lv.), number of categories in the 1st level (L1), total failure types (Total), whether it provides failure distribution (Dist.) and improvement strategies (Improv.). SAS and MAS indicates the type of LLM agents, i.e., single agent system (SAS) and multi-agent system (MAS).

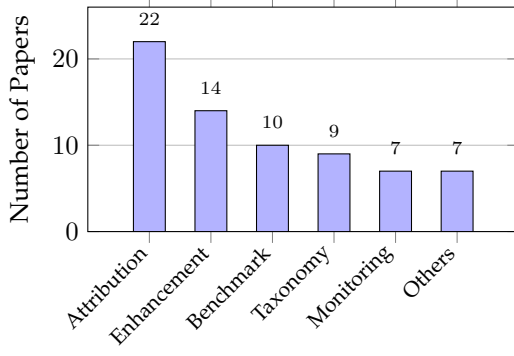


Fig. 7: Distribution of research dimensions in collected studies (note: papers may be assigned multiple topics)

(1) Taxonomies Organized by Task-Execution Phase (WHERE). This perspective dissects agent failures by following the natural, sequential pipeline of a task. It asks the fundamental debugging question: “Where in the task pipeline did it go wrong?” This perspective is highly intuitive for developers and aligns with operational monitoring, as it directly maps failures to observable phases in a task trajectory.

For example, Lu et al. [22] and Guo et al. [23] both adopt this phase-based lens to classify failures. Lu et al. develop a three-tier taxonomy of failure causes, highlighting planning errors, task execution issues, and incorrect response generation. Specifically, task planning related failures include improper task decomposition, failed self-refinement leading to infinite loops, and unrealistic planning that exceeds agent capabilities. Task execution failures include failure to exploit external tools, production of flawed code, and improper environmental setup. Response generation phase is prone to formatting issues, context window restraint causing information loss, and exceeding the maximum interaction rounds.

(2) Taxonomies Organized by Agent Capability Module (WHAT). Moving from external workflow to internal cognition, this perspective focuses on the agent’s core functional components. It investigates: “What core cognitive function of the agent failed?” Taxonomies such as AgentErrorTaxonomy [24] and TRAIL [25] adopt this view, organizing failures around modules like memory, reflection, planning, action, and system execution. This perspective is essential for research aimed at enhancing specific agent capabilities and understanding the compositional nature of agent intelligence.

For example, Zhu et al. [24] introduce the AgentErrorTaxonomy, a modular classification of failure modes spanning memory, reflection, planning, action, and system-level operations. Memory failures indicate the errors in recalling or retrieving information (false recall, omission, retrieval failure) that distort subsequent reasoning. Reflection failures are those in monitoring progress or interpreting outcomes (progress misassessment, outcome misinterpretation), blocking course correction. Planning failures include logically unsound or infeasible strategies (impossible actions, constraint ignorance, incoherent subgoals) that cascade into missteps. Action failures are

mistakes in executing operations (malformed outputs, incorrect parameters, missing arguments) that are visible but often mask upstream errors. System-level failures are those outside reasoning, such as tool crashes, API mismatches, or exceeding step limits, highlighting system robustness issues.

(3) Taxonomies Organized by System & Interaction (WHO). This perspective scales the analysis to the level of agent systems and their ecosystem. It addresses failures emergent from interactions, asking: “Who is responsible - the agent, the coordination, or the platform?” Frameworks like MAST [28], AgentFail [27] and AgentRx [26] employ this perspective, with top-level categories like specification issues, inter-agent misalignment, and platform-level failures. This view is critical for diagnosing the complex, often non-deterministic failures that arise in multi-agent collaboration, where the problem lies not in any single node’s reasoning but in the communication, shared understanding, or infrastructure governing their collective behavior.

For example, Cemri et al. [28] present MAST (Multi-Agent System Failure Taxonomy), and identify 14 unique failure modes, organized into 3 categories: specification issues, inter-agent misalignment, and task verification. Specifically, specification failures originate from system design decisions, and poor or ambiguous prompt specifications. Inter-agent misalignment failures arise from breakdowns in inter-agent interaction and coordination during execution. Task verification failures involve inadequate verification processes that fail to detect or correct errors, or premature termination of tasks.

(4) Taxonomies Organized by Environment Context (WHY). This perspective focuses on failure modes shaped by the environmental conditions under which the agent operates. It asks: “Why do failures arise under a particular environment context?” Such taxonomies are useful when properties of the surrounding environment, such as limited observability, stateful navigation, or turn/token constraints, strongly influence agent behavior and failure patterns.

The taxonomy in Aegis-Song [29] is representative of this perspective. It characterizes how agents fail when interacting with their environments, including exploration failures, exploitation failures, and resource exhaustion. For example, exploration failures occur when the agent cannot gather the critical context required from the environment; exploitation failures arise when the agent has obtained relevant information but fails to use it correctly under environmental constraints; and resource exhaustion captures failures induced by limited turns or token budgets.

4.1.2 Most Prevalent Failures

Table 2 lists the top three failures specified in each failure taxonomy. We consolidate the the most frequently occurring failure types across various studies, as follows.

Instruction & Format Fidelity: This type, manifested as formatting errors (TRAIL), response content deviation (AgentFail), instruction adherence (AgentRx), and instruction non-compliance (TRAIL, Aegis-Song), is top-ranked. It highlights a persistent struggle to reliably follow output specifications and complex instructions, a

fundamental limitation that has also been consistently identified in broader LLM research [76], [77].

Task Planning & Decomposition: Failures like Improper Task Decomposition (Lu et al.), Inefficient Plan (AgentErrorTaxonomy), and Goal Deviation (TRAIL) point to the shortcomings in logically structuring and adhering to long-horizon tasks.

Interaction, State & Coordination: In multi-agent settings, Step Repetition and Reasoning-Action Mismatch (MAST) are prevalent, exposing difficulties in maintaining coherent state and collaboration.

Knowledge & Reasoning Limits: Knowledge or Reasoning Limitation (AgentFail) and Hallucination (implied in many) remain fundamental constraints of the underlying LLMs.

4.2 Failure Attribution

Failure attribution is the cornerstone for diagnosing and improving LLM agents. It aims at systematically determining which agent is at fault, at which step the critical error occurred, and why. The central challenge lies in reasoning backwards from observed failures within complex, heterogeneous interaction data (spanning LLM reasoning, tool calls, and environmental feedback), to infer the internal logic breakdown and pinpoint the root cause. This task is particularly difficult because the point where a failure is observed is often not its origin. Initial errors in planning, knowledge, or coordination can propagate and transform through multiple steps of agent reasoning and tool execution, obscuring their original source. This propagation creates a significant gap between symptoms and root causes, demanding more sophisticated, inference-based attribution techniques.

Current research on failure attribution for LLM agents can be broadly organized into four categories, reflecting a progression in analytical depth and intervention intensity. We begin with pattern analysis-based approaches (Section 4.2.1), which passively mine statistical regularities from execution traces to localize faults. Building on observational data, LLM reasoning-based methods (Section 4.2.2) leverage the innate analytical capabilities of foundation models to simulate human-like diagnostic reasoning. To overcome the limitations of general-purpose LLMs, model fine-tuning-based techniques (Section 4.2.3) train specialized tracer models on curated failure datasets, aim at achieving greater efficiency and accuracy. Finally, dynamic runtime-based approaches (Section 4.2.4) adopt an interventionist paradigm, actively testing causal hypotheses through runtime edits and simulations. We introduce each category in detail in the following subsections.

4.2.1 Pattern Analysis-Based Attribution

As the most observational approach among attribution techniques, pattern analysis-based methods localize faults by mining recurring signals across numerous execution traces, relying purely on data-driven statistical analysis without runtime intervention. This family of techniques has evolved through three progressive stages: from coarse suspiciousness scoring, to stage-level accountability tracing, and finally to structured error representation with causal awareness.

The earliest works adapted classic fault localization ideas from traditional SE to the agent setting. FAMAS [30] pioneered this technical line by adapting spectrum-based fault localization. It systematically replaying trajectories to compute a “suspiciousness” score for each agent action based on its frequency in failing versus successful runs, with a tailored formula integrating both agent-level and action-level activation patterns. Barrak et al. [31] approached the same goal through a Planner → Executor → Critic pipeline, where blame propagates forward until the earliest unrepaired error is pinpointed.

Subsequent works shifted focus toward knowledge reuse and efficiency. CORRECT [32] distills recurring failures into compact error schemata that are reused for real-time recognition, decoupling diagnosis from costly trajectory replay. SDBL [33] addresses the scalability challenge by narrowing the search space before detailed localization, drawing on code defect manifestation patterns to scope potential fault regions.

The most recent works enrich pattern-based attribution with structured error modeling. AgentEval [23] constructs a directed acyclic graph (DAG) to explicitly track how root causes propagate through subsequent steps, moving beyond point-wise localization to capture error cascades. PROMAS [34] introduces causal delta representations with a vector Markov entropy model, pinpointing the exact moment a trace diverges from successful behavior-effectively combining pattern matching with causal change detection.

4.2.2 LLM Reasoning-Based Attribution

LLM reasoning-based approaches move from mining correlations to simulating human-like diagnostic reasoning. These methods leverage the analytical capabilities of foundation models to infer root causes from complete task trajectories through sophisticated prompting and structured reasoning processes. The evolution within this category follows a clear trajectory: from naive prompting, to structured reasoning pipelines with consensus mechanisms, to explicit causal modeling, and finally to hybrid approaches that augment LLM reasoning with external verification or memory.

The earliest instantiation of this paradigm used straightforward prompting over complete trajectories. The pioneering Who&When benchmark [11] established this paradigm by evaluating three basic strategies, i.e., All-at-once, Step-by-step, and Binary search, demonstrating the feasibility but also the limited accuracy of using LLMs on this complex task.

To enhance reliability, subsequent work introduced more structured reasoning processes. ECHO (Error attribution through Contextual Hierarchy and Objective consensus analysis) [35] develops a hierarchical context representation of the entire interaction trace, provides independent objective analyses across these contexts and cross-validating the findings via consensus voting. RAFFLES [36] adopts an iterative, multi-component pipeline where a central Judge systematically investigates faults while specialized Evaluators assess not only system components but also the quality of the Judge’s own reasoning, creating a self-correcting diagnostic loop.

TABLE 3: Overview of failure attribution studies in LLM agents

Category	Approach	Main idea	Passive/ Active	Statistical/ Reasoning	General/ Specialized	Post-hoc/ Runtime
Pattern Analysis Based	FAMAS [30]	Spectrum-based analysis	Active	Statistical	General	Post-hoc
	CORRECT [32]	Pattern distillation + real-time recognition				
	Barrak’s [31]	Planner → Executor → Critic pipeline	Passive			
	SDBL [33]	Scope delineation leveraging code defect patterns				
	AgentEval [23]	DAG for error propagation tracking				
	PROMAS [34]	Causal delta + vector Markov entropy				
LLM Reasoning Based	Who&When [11]	Three prompting strategies	Passive	Reasoning	General	Post-hoc
	ECHO [35]	Hierarchical context + Consensus voting				
	RAFFLES [36]	Central judge + iterative refinement				
	CDC-MAS [37]	Multi-granularity causal inference				
	A2P [38]	Abduce, intervene, verify				
	CHIEF [39]	Hierarchical causal graph				
	AgentRx [26]	Constraint synthesis + LLM judgement				
	CodeTracer [40]	Hierarchically structured trace tree				
	ERRORPROBE [41]	Symptom-driven backward tracing + episodic memory				
Model Fine- Tuning- Based	AgentTracer [42]	Multi-granular reinforcement learning	Passive	Reasoning	Specialized	Post-hoc
	GraphTracer [43]	Information dependency graphs				
	Aegis-Kong [44]	Three learning strategies (SFT/RL/Contrastive)				
Dynamic Runtime- Based	DoVer [45]	Intervention-driven debugging	Active	Reasoning	General	Runtime
	AgentDebug [24]	Self-diagnosis + re-rollout				
	TraceElephant [46]	LLM agent + counterfactual verification				
	AgentFail [27]	LLM candidate + counterfactual verification				

Notes: 1) *Passive/Active*: Passive methods analyze existing trajectories without extra execution; Active methods generate additional traces or intervene during execution. 2) *Statistical/Reasoning*: Statistical methods rely on spectrum, pattern, or entropy models; Reasoning methods leverage LLM-based inference (including fine-tuned models). 3) *General/Specialized*: General methods are model-agnostic; Specialized methods require fine-tuning a specific base model (e.g., Qwen, Llama). 4) *Post-hoc/Runtime*: Post-hoc methods analyze logged traces after task completion; Runtime methods perform attribution during execution or involve re-execution/intervention.

A further wave of methods explicitly models causality within the reasoning process. CDC-MAS [37] introduces a performance causal inversion principle that reverses data flow in execution logs, combined with Shapley values for agent-level blame assignment, and a causal discovery algorithm robust to non-stationary interaction data. A2P [38] guides the LLM through a formal three-step process-Abduction (infer hidden root causes), Action (define corrective intervention), and Prediction (verify whether the intervention resolves the failure), all within a single inference pass, tightly coupling diagnosis with repair hypothesis. CHIEF [39] constructs hierarchical causal graphs to trace fault propagation, then employs oracle-guided backtracking to prune the search space and progressive causal screening to distinguish true root causes from propagated symptoms, achieving the highest attribution accuracy among reasoning-based methods.

Beyond structural and causal augmentation, some methods incorporate external mechanisms to ground LLM reasoning. CodeTracer [40] organizes execution logs into a hierarchically structured trace tree with nested scopes, enabling the LLM to reason at appropriate abstraction levels. AgentRx [26] combines constraint synthesis with LLM-based judgement, producing auditable validation logs of constraint violations that the LLM then localizes. ERRORPROBE [41] augments reasoning with episodic memory, using retrieval-augmented verification to mitigate hallucination in the diagnostic process itself.

4.2.3 Model Fine-Tuning-Based Attribution

Unlike prompting-based methods, this paradigm focuses on training specialized models dedicated to the failure

attribution task. The resulting specialized tracer model learns to efficiently parse complex agent interactions, recognize error propagation patterns, and localize faults with improved accuracy and inference efficiency compared to general-purpose LLM.

AgentTracer [42] established a foundational pipeline for this direction. It first annotates failed agent trajectories via counterfactual replay and programmed fault injection, producing the curated dataset TracerTraj. Leveraging this resource, they develop AgentTracer-8B, a lightweight failure tracer trained with multi-granular reinforcement learning, capable of efficiently diagnosing errors in verbose multi-agent interactions.

GraphTracer [43] moves beyond sequential trajectory analysis to a structured graph representation. It constructs Information Dependency Graphs (IDGs) to explicitly capture how agents reference and build on prior outputs, and to enable tracing fault propagation along dependency edges rather than temporal steps. The training data is produced through graph-aware fault injection, and the failure tracer also trained guided by graph-structural rewards.

Subsequently, Aegis-Kong [44] scaled and systematized this paradigm. It created one of the largest synthetic attribution datasets to date and performed a comprehensive empirical study, directly comparing the efficacy of three distinct learning strategies, i.e., supervised fine-tuning, reinforcement learning, and contrastive Learning, for training robust failure attribution models. The results showed that reinforcement learning and contrastive learning outperform

standard supervised fine-tuning, suggesting that the attribution task benefits from objectives beyond next-token prediction.

4.2.4 Dynamic Runtime-Based Attribution

Moving beyond passive log analysis, this group of techniques adopts an active, interventionist approach to attribution. The key design principle is to formulate hypotheses about the root cause and then test them through targeted interventions, i.e., systematically testing whether modifying a hypothesized root cause would have averted the failure.

DoVer [45] is an intervention-driven debugging framework, which augments hypothesis generation with active verification through targeted interventions (e.g., editing messages, altering plans). AgentDebug [24] applies a similar idea by assigning fine-grained error types to each step and module, and then guiding targeted re-rollouts based on the diagnosed error types.

The paradigm further evolves to tightly integrate LLM reasoning with dynamic verification. Works like TraceElephant [46] and AgentFail [27] exemplify this by employing an LLM-based candidate identification step followed by counterfactual verification.

4.2.5 Performance Comparison

Figure 8 demonstrates the comparative overview of failure attribution performance in terms of all investigated techniques. Most techniques were evaluated on the Who&When benchmark [11], making their results directly comparable. But note that, different methods And the comparison is organized into hand crafted systems and algorithm generated systems following the benchmark’s original setup.

Generally speaking, agent-level accuracy is significantly higher than step-level accuracy, which is well understood because identifying the responsible agent is inherently easier. For algorithm-generated systems, step-level accuracy tends to be higher than for hand-crafted systems, likely because algorithm-generated systems follow more consistent patterns, making it easier to discern underlying regularities, whereas hand-crafted trajectories tend to be longer and more variable, increasing the difficulty of accurate step-level attribution. Furthermore, fine-tuning-based methods achieve relatively better performance, as they leverage additional training data that enables the model to learn the organizational patterns of trajectories and the failure modes. Additionally, some LLM reasoning-based approaches, such as CHIEF and A2P, also deliver competitive results through detailed causal analysis of trajectories.

In terms of reported accuracy, on the Who&When benchmark, reported step-level attribution accuracy varies substantially across techniques, ranging from about 25% [11] to 52% [39]. Meanwhile, TraceElephant is a more recent benchmark and has so far been evaluated by relatively fewer methods; the currently reported step-level accuracy is about 33% [46]. This indites the step-level attribution accuracy is still limited. Also note that, the comparability across studies remains limited due to differences in datasets, observability settings, and evaluation protocols.

4.3 System Enhancement and Optimization

The ultimate goal of diagnosing failures is to enable improvement. Once the root cause of a failure is attributed, the next critical step is to leverage this insight to enhance the system’s capabilities, reliability, and efficiency. This section focuses on approaches that utilize trajectory analysis to drive the enhancement and optimization. This can be achieved by reshaping the external structure and environment in which agents operate (Section 4.3.1), by innovating the internal drivers and mechanisms of the agents themselves (Section 4.3.2), or by adding a lightweight layer for process management or supervision at runtime (Section 4.3.3).

4.3.1 Structural & Workflow Optimization

This paradigm focuses on enhancing overall performance and robustness by re-designing the collaborative architecture and interactive environment of multi-agent systems from the outside. Existing works can be organized by the depth of their intervention, targeting progressively deeper layers of the system stack.

Environment-level optimization represents the shallowest form of intervention, modifying only the conditions under which agents operate without touching their core logic. Song et al. [29] design Aegis, which enhances environment observability, offloads common computations, and speculates agentic actions.

Workflow-level optimization moves the target inward to the collaborative graph structure and its associated configurations. Wang et al. [47] introduce Maestro, a framework-agnostic optimizer that jointly searches over graph topologies and agent configurations under explicit rollout and token budgets, guided by reflective textual feedback from execution traces. CE-Graph [48] adopts a diagnosis-driven variant: it first estimates the failure distribution from counter-examples, identifies dense failure regions as recurring modes, and then applies targeted, operator-constrained graph edits. ILWS [49] pushes this level further by treating system-level instructions as versioned, editable pseudo-parameters that are evolved by a reflection engine after each session. Unlike Maestro and CE-Graph which optimize the graph topology, ILWS refines the shared guiding principles that govern all agent behavior, enabling post-hoc self-improvement without modifying the underlying models.

4.3.2 Agent Internal Optimization

In contrast to modifying external frameworks, this line of research focuses on optimizing agents’ internal decision-making logic to enhance their performance. Approaches can be organized by the granularity of what they modify: from high-level prompts down to model mechanism and reusable skills.

Prompt-level optimization is the least invasive. SCOPE [50] frames prompt evolution as an online learning problem, balancing tactical error resolution with strategic principle refinement. Fangs [69] takes a complementary, attribution-driven approach: it first identifies which decisions or reasoning steps caused a failure, then generates contextaware guidance to steer future behavior.

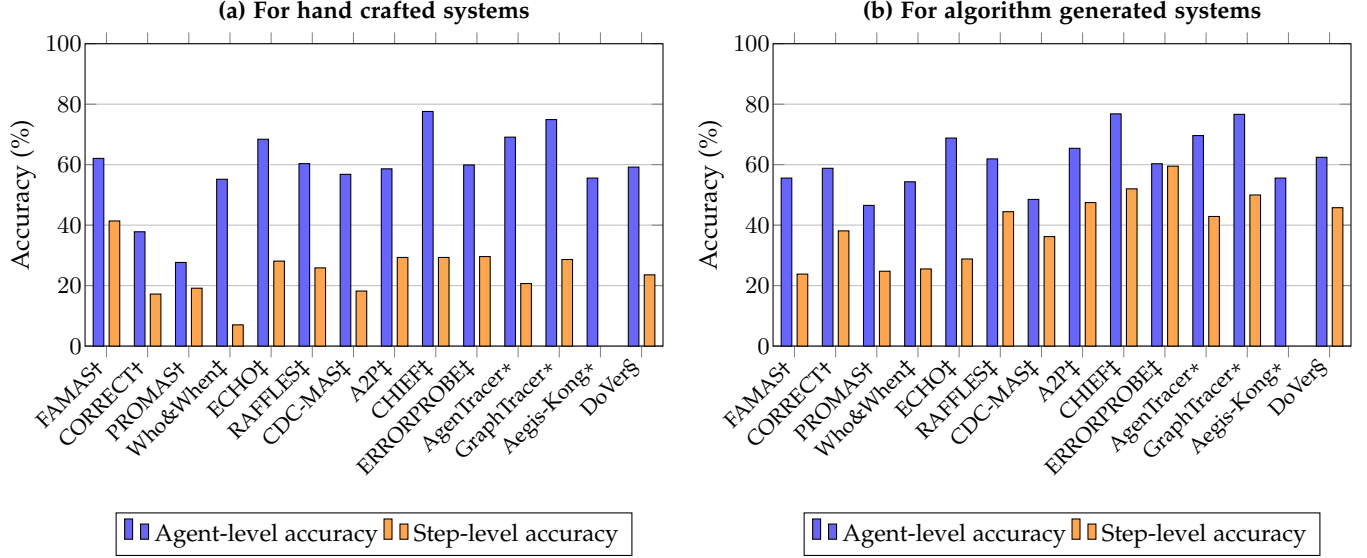


Fig. 8: Failure attribution performance reported in collected studies. Missing bars indicate no reported result for that metric. All these methods are evaluated on the Who&When benchmark (“With Ground Truth” setting). Approaches not shown in this figure adopt other evaluation metrics or evaluated in other benchmarks. Approach categories are indicated by symbols: †= Pattern Analysis, ‡= LLM Reasoning, * = Model Fine-Tuning, §= Dynamic Runtime.

TABLE 4: Enhancement and optimization approaches for LLM agents

Category	Approach	Objec.	Main idea	Timing	Level	What to modify
Structural & Workflow Optimization	Aegis-Song [29]	F + E	Rule-driven environment tuning (enhance observability, offload common computations, speculate agentic actions)	OL	Agent environment interaction	Environment configuration
	Maestro-Wang [47]	F	Graph search + reflective textual feedback	TD	System + Agent	Workflow structure + agent configuration
	CE-Graph [48]	F + E	Graph editing via propose-and-verify based on estimated failure distribution	TD	System	Workflow structure
	ILWS [49]	F + E	Instruction shaping + Feedback learning	OL	System + Agent	Prompts
Agent Internal Optimization	SCOPE [50]	F + A	Prompt evolution + Online learning	OL	Agent	Prompts
	AgentDevel [51]	F	Flip-centered gating with executable diagnostic scripting	TD	Agent	Instructions + Policies
	ReCreate [52]	F + E	Agent-as-optimizer leveraging interaction trajectories	TD	Agent	Instructions + Policies
	SEAlign [53]	F	Tree search for fine-grained alignment and preference optimization on critical actions	TD	LLM	LLM
	Fang’s [69]	F	Attribute failures to specific decisions and reasoning steps, generate context-aware guidance	TD	Agent	Prompts
	Trace2Skill [55]	F	Parallel multi-agent patch proposal and conflict-free consolidation	TD	Agent	Skill
	AgentDiet [56]	E	Rule-driven trajectory compression	OL	Interaction	Interaction data
Runtime & Supervisory Optimization	SUPERVISOR AGENT [57]	F + E	Proactive error correction, inefficiency guidance, adaptive observation purification	OL	Supervisory layer	External supervision policy
	Wink [58]	F	Runtime periodic intervention to nudge agents out of misbehavior	OL	Supervisory layer	Interaction data
	Liu’s [54]	F + E	Intervention based on process-centric metrics	OL	Supervisory layer	Interaction data

Note: Objec. indicates the optimization objective. F, E, and A in Objec. column are short for Functionality, Efficiency, and Adaptability in new scenarios; OL and TD in Timing column respectively short for Online and Trace-driven.

Policy/instruction-level optimization modifies the agents internal policies or system instructions, e.g., avoid tool calls without verification. AgentDevel [51] formalizes self-evolution as a release engineering pipeline: an LLM critic characterizes failures, script-based diagnosis produces engineering specifications, and flip-centered gating prioritizes regression fixes. ReCreate [52] leverages interaction trajectories as signals to automatically create and enhance domain agents, turning episodic outcomes into persistent updates to the agents behavior rules.

Model-level optimization directly adjusts the underlying LLM. SEALign [53] uses tree search to perform fine-grained preference optimization on critical actions, directly adjusting the models decision preferences rather than surface prompts.

Skill-level optimization builds reusable capabilities. Trace2Skill [55] proposes parallel patches from multiple trajectories and merges them via conflict-free consolidation, transforming episodic fixes into composable skills.

4.3.3 Runtime & Supervisory Optimization

The third paradigm instead enhances efficiency and robustness by optimizing the data flow of the execution process or attaching a lightweight external supervisory layer.

Input-space optimization reduces noise in the agents context. AgentDiet [56] removes useless, redundant, or expired information from trajectories, cutting token cost without altering decision logic.

Behavior-space optimization injects external steering signals during execution. SUPERVISORAGENT [57] uses an LLM-free adaptive filter to detect critical junctures and proactively correct errors. Wink [58] prioritizes low overhead by periodically nudging agents out of misbehavior, making it suitable for continuous background monitoring. Lius [54] triggers interventions based on process-centric metrics (e.g., progress rate, action entropy), enabling adaptive, goal-driven steering.

4.3.4 Performance Comparison

Table 5 provides an overview of the reported performance gains of different enhancement approaches. Note that, these methods are evaluated on different tasks, benchmarks, and base models, making a strictly fair comparison impossible. Nevertheless, the table serves as an intuitive reference for readers to quickly grasp the magnitude of improvements each method claims in its own setting.

Some approaches are evaluated with different metrics, and not included in the table. For example, runtime & supervisory optimization approaches, like AgentDiet [56] and SUPERVISOR AGENT [57] are evaluated with input token reduction, achieving 39.9% - 59.7% and 29.45%, respectively, while preserving task performance.

4.4 Trajectory Monitoring and Analysis Tools

The observability of an agent system, i.e., enabling the capture and understanding of its internal states and interactions, is essential for failure diagnosis and system improvement. This section reviews approaches that enhance observability by making execution traces interpretable, allowing both automated analysis and human-in-the-loop inspection.

The works in this area can be broadly grouped by their primary focus: tools for low-level, non-invasive system monitoring and passive diagnosis (Section 4.4.1), versus those for high-level, interactive analysis and active debugging (Section 4.4.2).

4.4.1 System-Level Monitoring and Passive Diagnostic Tools

This category targets at the passive, transparent, and comprehensive collection of execution data from agent systems.

The first set of tools focuses on external, non-invasive instrumentation that observes agent behavior without modifying application code. AgentSight [59] is an observability framework which monitors agents from outside their application code at stable system interface, which is used for identifying resource-wasting reasoning loops, revealing hidden coordination bottlenecks.

Building on passive data collection, another line of work embeds structured diagnostic analysis directly into the monitoring pipeline. Moshkovich et. al. [60] present the AgentOps automation pipeline, a six-stage process encompassing behavior observation, metric collection, issue detection, root cause analysis, optimized recommendations, and runtime automation.

A third direction prioritizes human interpretability, building interactive visualizations that make trajectories accessible to developers. AgentDiagnose [61] provides a framework for quantitative visualization and diagnose. It has (i) an evaluation module that quantifies five core agentic competencies, i.e., backtracking and exploration, task decomposition, observation reading, self-verification, and objective quality, and (ii) a visualization module that highlights trajectory semantics through t-SNE action embeddings, interactive word clouds, and state-transition timelines. Similarly, Agent Trajectory Explorer [62] provides an intuitive interface to support visual inspection and manual annotation of agent execution trajectories.

4.4.2 Interactive Analysis and Active Debugging Tools

Moving beyond data collection, this category focuses on building interactive tools that empower developers to explore and intervene in agent trajectories.

AGDebugger [63] is an interactive debugging tool, which allows users to reset to previous states and modify past agent messages, so as to support iterative, hypothesis-driven debugging. XAgent [64] bridges automated judgment with human-in-the-loop feedback. It can turn execution traces into an interactive flowchart for trajectory visualization, and also allows users to annotate outputs which serves as feedback for failure attribution and iterative improvement. DiLLS [65] is designed to reveal and structure the behaviors of multi-agent systems, and it organizes execution information into three layers, i.e., overall planning and execution, action arrangement and outcomes, and logs for concrete operations.

4.4.3 Practitioner-oriented Observability Platforms

Beyond research prototypes described in the literature, the broader ecosystem also includes widely used practitioner-oriented observability platforms for tracing and analyzing

TABLE 5: Performance improvements of enhancement approaches on various tasks

Method	Task success rate	Base Model	Tasks
Aegis-Song † [29]	Relative improvement + 6.7% – 12.5%	GPT-4.1	Airline, retail, file sys, CRM, medical
Maestro-Wang † [47]	Relative improvement + 65.7%	gpt-4o-mini, gpt-4.1-mini, gpt-4.1-nano	Interviewer agent, AI assistant
CE-Graph † [48]	From 77.50% to 86.23% (+8.73%)	gpt-4o-mini	HumanEval, MBPP, GAIA, etc.
SCOPE ‡ [50]	From 14.23% to 38.64% (+24.41%)	GPT-4.1	HLE, GAIA, DeepSearch
AgentDevel ‡ [51]	From 15% to 30% (+15%)	Claude-Sonnet-4.5	SWE-bench Verified
ReCreate ‡ [52]	From 59.29% to 66.15% (+6.86%)	claude-4.5-opus	SWE-bench Verified, AppWorld, DS-1000, etc.
SEAlign ‡ [53]	From 3.7% to 17.7% (+14.0%)	14B Qwen	SWE-Bench-Lite
Fang’s ‡ [69]	From 69.6% to 73.2% (+3.6%)	GPT-4.1	AppWorld
Trace2Skill ‡ [55]	From 31.57% to 46.35% (+14.78%)	35B Qwen	SpreadsheetBench-Verified, WikiTable-Questions
Liu’s § [54]	From 11.8% to 35.3% (+23.5%)	DeepSeek R1	SWE-bench Verified

Note: † = Structural & Workflow Optimization, ‡ = Agent Internal Optimization, § = Runtime & Supervisory Optimization. Approaches not shown here adopt other evaluation metrics. For papers (e.g., Aegis-Song) where absolute values are from figures (not precisely readable), we report only relative improvement as stated in the text.

TABLE 6: Datasets and benchmarks for failure analysis

Data Type	Benchmark	Source Systems	Scale	Annotation	Adoption	Usage	Specialty
RW	Who&When [11]	CaptainAgent (GAIA, AssistantBench), Magnetic-One (GAIA, AssistantBench)	127	Responsible agent, faulty step, some with root cause	High	Eval.	First benchmark with clear data organization
	TRAIL [25]	OpenDeepResearch (GAIA), CodeAct agent (SWE-Bench Lite)	148	Coarse and fine grained root cause (error type, location)	Low	Eval.	
	AgentError Bench [24]	N/A (ALFWorld, GAIA, WebShop)	200	Failure step, coarse and fine grained root cause (module and error type)	Low	Eval.	
	TraceElephant [46]	CaptainAgent (GAIA, AssistantBench), Magnetic-One (GAIA, AssistantBench), SWE-Agent (SWE-Bench verified)	220	Responsible agent, faulty step	Low	Eval.	Full observable data and running environment
	AgentFail [27]	10 systems from Dify and Coze platform (e.g., HumanEval, SWE-Bench, TravelPlanner)	307	Responsible agent, root cause, repair strategy	Low	Eval.	
	AgentRx [26]	7-bench, Flash, Magentic-One	115	Failure step, root cause (failure label), first unrecoverable failure	Low	Eval.	Marks the first unrecoverable failure
	CodeTrace Bench [40]	SWE-Agent, MiniSWE-Agent, OpenHands, Terminus 2	4354	Responsible stage (e.g., debugging, patching), faulty step, failure reason (supporting evidence)	Low	Trn./Eval.	
	MP-Bench [66]	CaptainAgent (GAIA, AssistantBench), Magnetic-One (GAIA, AssistantBench)	289	faulty step, failure reason, ideal action	Low	Eval.	Attribution from multiple perspectives
EI	Aegis-Kong [44]	6 systems (Magnetic-One, MacNet, DyLAN) from 6 task sources (GAIA, MATH, HumanEval)	9,533	Responsible agent, root cause (error mode)	Low	Trn./Eval.	
	CORRECT-Error [32]	Magnetic-One and AutoGen (GAIA, Math500, MMLU-Pro, HotpotQA)	2,000+	Faulty step	Low	Trn./Eval.	

Note: RW and EI refer to Real-World failures and synthetic data by Error Injection respectively; Eval. and Trn. refer to Evaluation and Training. Scale indicates the number of failure trajectories. Adoption indicates the extent of usage by other studies.

LLM agent trajectories. Representative open-source examples include Langfuse², Weave³, and Arize Phoenix⁴, which support trace inspection, prompt/tool-call monitoring, evaluation, and workflow debugging. There are also commercial or hosted platforms, such as LangSmith⁵, HoneyHive⁶, and

Braintrust⁷, that provide production-facing tracing, evaluation, and monitoring capabilities for multi-step LLM applications and agentic systems. Although these tools are not introduced through archival research publications and are therefore outside our main survey corpus, they play an important role in practical AgentOps and can serve as valuable infrastructure for trajectory monitoring and diagnosis in real-world deployments.

2. <https://langfuse.com/>

3. <https://github.com/wandb/weave>

4. <https://arize.com/phoenix/>

5. <https://www.langchain.com/langsmith/observability>

6. <https://www.honeyhive.ai/>

7. <https://www.braintrust.dev/>

4.5 Datasets and Benchmarks

Benchmarks and datasets provide foundational support for research in this field: they serve as objective, unified evaluation standards for measuring and comparing the performance of different attribution techniques, and as sources of high-quality training data for powering data-driven diagnostic models. Existing studies can be broadly categorized into two types: evaluation-centric benchmarks collected from real-world failures, and synthetic, error-injected datasets designed for both training and evaluation.

4.5.1 Benchmarks from Real-World Failure Collection

These datasets collect naturally occurring failures from actually deployed agents, thus closely reflecting real-world application scenarios.

Zhang [11] introduce the Who&When benchmark, the pioneering benchmark in this field, being utilized by almost all failure attribution studies. Each failure trajectory is annotated with failure-responsible agent, the corresponding failure step, and some with the reasons for failure in natural language. AgentRx [26] focuses on the first unrecoverable failure step, arguing that recoverable steps are less meaningful for attribution. Chen et al. [46] note that existing benchmarks rely on partial observability (agent outputs only), so they propose TraceElephant with full execution traces and reproducible environments. This aligns with real-world developer facing scenarios where complete traces, rather than only outputs, are accessible for diagnosis.

Other benchmarks offer finer-grained or multi-perspective annotations. TRAIL [25] and Agent Error Benchmark [24] provides finer-grained root cause, to facilitate the better understanding of the failures. MPBench [66] introduces multi-perspective attribution by annotating an ideal action alongside the faulty step and failure reason, enabling richer diagnostic signals.

Several benchmarks target specific agent types rather than generic LLM agents. AgentFail [27] focuses on agentic workflow agents, providing fine-grained, lifecycle-spanning annotations (precise location, root cause, and repair strategy), which enables deeper analysis of the entire failure process beyond simple error detection. CodeTraceBench [40] focuses on software engineering agents (SWEAgent, OpenHands, etc.).

4.5.2 Synthetic Datasets via Error Injection

These datasets are constructed by actively injecting errors into successful trajectories, resulting in precise error localization and annotation. Besides evaluation, they are also suitable for model training.

Kong et al. [44] construct Aegis, a large dataset of 9,533 trajectories with annotated faulty agents and error modes. This is achieved using a LLM-based manipulator that can adaptively inject context-aware errors into successful execution trajectories. Yu et al. [32] construct CORRECT-Error, a large-scale collection of over 2,000 multi-agent trajectories with fine-grained, step-level error annotations. It is built using a error-injection pipeline guided by real-world failure distributions, yielding trajectories that balance controlled coverage with the realism of natural MAS failures.

5 DISCUSSION

5.1 Gap Between Attribution to Enhancement

Figure 9 visually summarizes the outcomes of each failure attribution technique, as well as the income of each enhancement technique.

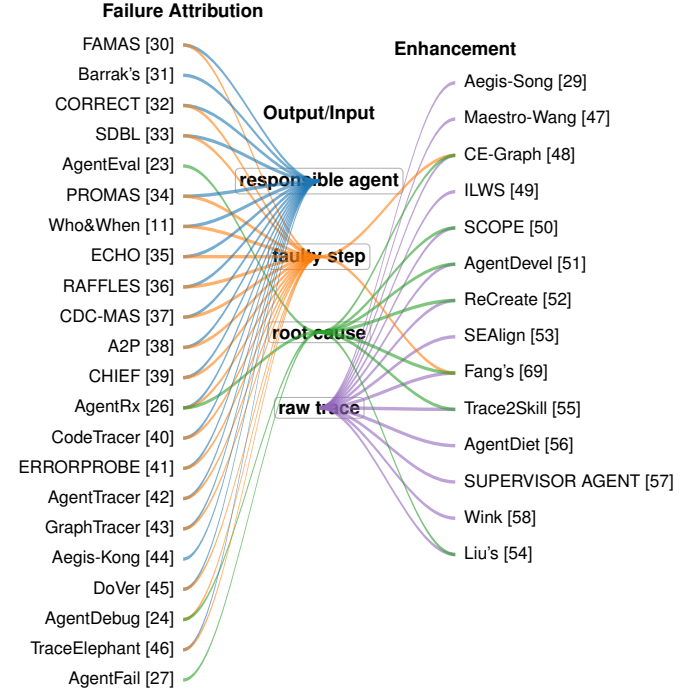


Fig. 9: Sankey diagram of the relationship between the output of failure attribution techniques and input of enhancement techniques.

We can observe that failure attribution techniques predominantly output three types of information, i.e., responsible agent, faulty step, and root cause, with responsible agent being the most common. On the enhancement side, nearly all approaches rely on raw traces as their primary input, while only a few accept faulty step or root cause.

A clear gap emerges: the widely produced responsible agent information is never consumed by any enhancement method. This may be because such coarse-grained information (i.e., which agent is responsible) provides insufficient detail about exactly where the agent went wrong or how to fix it, and therefore offers little actionable guidance for subsequent repair. Even for faulty step and root cause, the flow from attribution to enhancement is sparse, with only a handful of techniques, such as CEGraph [48] and Fang's [69], utilizing them.

The limited use of *faulty step* can be explained by the fact that a faulty step merely indicates a specific position within an execution trace, but does not directly point to which part of an agent should be modified. Consequently, it cannot provide targeted guidance for enhancement techniques. By contrast, *root cause* is adopted by a larger number of techniques. This is because many enhancement techniques rely on the cause or description of the failure to infer appropriate modification strategies. Nevertheless, not all techniques leverage the failure analysis information and some techniques optimize only on final task scores, success rates,

or other aggregated data, without performing any failure cause analysis.

Overall, the pipeline from failure attribution to system enhancement is broken: attribution focuses on identifying responsible agents, but enhancement techniques are not designed to consume that information, and they still rely on raw traces rather than the structured outputs of attribution.

5.2 Guidance for Method Selection

While the previous sections have organized existing techniques along multiple dimensions, practitioners seeking to apply these methods in real-world scenarios may face high-level decisions, such as whether the target environment is production or research, what budget constraints (e.g., token consumption, training overhead) apply. This subsection synthesizes the current evidence to try to offer actionable guidance.

Production vs. research environments. Several approaches originate from industrial research labs (e.g., ECHO [35] from Amazon Web Services, AgentRx [26] from Microsoft Research). However, their evaluations are almost exclusively conducted on academic benchmarks such as Who&When, using agents (e.g., CaptainAgent, MagneticOne) that may not fully represent real production deployments.

Nevertheless, we have identified at least two methods that have been successfully deployed in production and report concrete operational benefits. Wink [58] (from Meta) resolves 90% of single-intervention misbehaviors in live A/B tests, and AgentDiet [56] (ByteDance) reduces token cost by 4060% while maintaining performance in the real coding-agent environment Trae Agent.

Budget constraints. we analyze the resource consumption patterns of different attribution techniques based on data reported in the original papers [39]. 1) LLM reasoning-based methods (e.g., CHIEF [39], ECHO [35]) incur a moderate token overhead, typically 2-3x compared to single-pass prompting. 2) The typical pattern-based methods FAMAS [30] involve repeated replay, leading to significantly higher token consumption (e.g., 6-8x more than reasoning-based approaches). 3) Fine-tuning-based methods (e.g., AgenTracer [42], GraphTracer [42]) can achieve lower inference cost per trace once deployed, but require substantial investment for model training. 4) Dynamic runtime-based methods (e.g., DoVer [45]) typically incur the highest per-diagnosis cost because they actively re-execute or intervene in the agent, and also demand online access to the execution environment. However, they often achieve the best attribution accuracy (as shown in Figure 8), as active hypothesis testing can disambiguate root causes more effectively than passive analysis.

Overall, no single method dominates across all dimensions. Practitioners should choose based on their accuracy requirements, budget (one-time vs. per-trace cost), and whether online intervention is feasible.

5.3 Connections to Established SE Techniques

The trajectory analysis of LLM agents is closely related to several established areas in software engineering. We map

TABLE 7: Connections between trajectory analysis of LLM agents and traditional software engineering techniques.

Survey Dimension	SE Analogue	Key Difference in LLM agent Setting
Failure taxonomy	Defect classification and taxonomies [81]	LLM agent failures often involve reasoning, coordination, and prompt/workflow issues, rather than only code-level defects.
Pattern-based attribution	Spectrum-based fault localization, log anomaly detection [19], [78]	Failure signals are language-heavy, probabilistic, and often less repeatable across runs.
Runtime debugging	Time-travel debugging, replay, breakpoints [82]	Replay is often non-deterministic, and runtime intervention is typically approximate rather than exact.
Monitoring	Distributed tracing, log observability [83]	Trajectories mix structured events with natural-language semantics, making tracing harder than in conventional traced systems.
Enhancement	Program repair, configuration tuning [79], [80]	Improvement targets often include prompts, orchestration logic, supervision policies, etc, rather than only source code.

several major dimensions of our survey to representative traditional SE techniques, as summarized in Table 7.

For example, pattern-based attribution is closely related to spectrum-based fault localization [78] and log anomaly detection [19], while enhancement and optimization methods are related to program repair and configuration optimization [79], [80]. At the same time, these analogies should not be overstated. Compared with traditional software systems, critical failure signals in LLM agents are often embedded in natural-language reasoning, runtime-generated interaction chains, and probabilistic decision processes. As a result, on the one hand, existing SE techniques can provide valuable conceptual and methodological foundations; on the other hand, their direct transfer to LLM agent settings is often non-trivial and requires substantial adaptation.

6 CHALLENGES AND OPPORTUNITIES

Based on the above analysis for trajectory related studies, we summarize the challenges and opportunities for future research.

6.1 Improving Attribution Accuracy via Enhanced Causal Understanding

Precise failure attribution remains a significant challenge due to the non-deterministic reasoning and intricate interactions inherent to LLM agents, with step-level accuracy on established benchmarks still limited (as shown in Figure 8). The difficulty lies in moving beyond surface-level trajectory inspection to the structured understanding of the agent’s internal decision logic. CHIEF [39], which achieves the highest performance so far, begin to address this by transforming flat execution logs into hierarchical causal graphs, explicitly modeling data transitions and error propagation. This demonstrates that formalizing the latent logical and causal

relationships within trajectories is a viable path to improved attribution.

This represents a valuable initial step in this research direction. Future exploration can develop more comprehensive models that can capture both the deterministic elements of an agent’s architecture (e.g., workflow structure, tool call protocols) and the probabilistic nature of its LLM-based reasoning. Furthermore, linking traditional software usually benefits from the combination of static analysis and dynamic running [84], [85], paradigm-breaking hybrid techniques might be another promising direction.

6.2 Rethinking Evaluation for LLM Agent Failure Analysis

The evaluation of failure attribution techniques must align with their intended use in debug and repair. We identify three key dimensions where current practices fall short.

6.2.1 From Accuracy to Utility: The Need for Repair-Driven Metrics

As shown in Section 5.1, failure attribution techniques predominantly output responsible agents, faulty steps, or root causes, yet enhancement methods rarely consume these structured results and they still rely on raw traces. This disconnect suggests that the current evaluation centered on step-level or agent-level accuracy may not reflect a techniques actual utility in enabling system repair.

Therefore, we advocate for repair-driven metrics that assess how well attribution outputs (e.g., root cause descriptions) can be directly used by enhancement methods or how much debugging effort they save. Such metrics would incentivize attribution techniques to produce information that is actually actionable, thereby helping to close the attribution-to-enhancement gap.

6.2.2 Beyond Partial Observability: The Demand for Comprehensive Benchmarks

Who&When [11] is almost the only benchmark specifically designed for failure attribution of LLM agents for a considerable period in the past. It provides partially observable execution traces that include only the agents outputs, without the corresponding inputs such as original task instructions, prompts, or contextual messages. This restricted setting can be suitable for evaluating attribution methods under certain black-box scenarios, where internal inputs or intermediate states are inaccessible (e.g., debugging deployed agents on external platforms). However, in many practical developer-facing debugging scenarios, developers typically have access to substantially richer execution information, including task instructions, prompts, intermediate messages, tool invocations, and environment states.

The recent introduction of benchmarks like TraceElephant [46], which offers complete execution traces and reproducible environments, addresses this gap and represents a necessary step forward. Future opportunities include creating more diverse, large-scale benchmarks from real-world deployments to robustly evaluate both attribution and enhancement methods under realistic conditions.

6.2.3 From Binary Outcomes to Capability Assessment: Evaluating in Open-Ended Scenarios

Most existing failure diagnostic methods provide only coarse-grained outputs, such as binary failure labels, and heavily rely on the availability of task-specific oracles (e.g., test cases) to determine success. However, LLM agents are increasingly deployed in open-ended scenarios (e.g., creative tasks) where clear oracles are scarce, moving beyond simple pass/fail judgments toward assessing agent capabilities in open-ended scenarios.

An initial step in this direction is demonstrated by AgentDiagnose [61], which provides a preliminary attempt to evaluate agents across five competency dimensions using LLM-as-a-judge. A critical opportunity lies in developing trajectory-based analysis that delivers fine-grained, multi-dimensional assessments of agent capabilities (e.g., planning fidelity, tool usage, state tracking). Such detailed diagnostics would not only aid developers in pinpointing weaknesses but could also serve as intermediate supervision signals to improve agent training.

6.3 Toward Systematic AgentOps: From Fragmented Tools to Integrated Governance

Despite progress in attribution, enhancement, and monitoring, these capabilities remain fragmented, i.e., there is no integrated framework to govern LLM agents from offline evaluation to production operation. Emerging industrial initiatives such as AgentOps SDK [86] and IBM Telemetry [87] signal the need for a systematic discipline, yet academic research lags behind. As pointed by a recent survey [88], AgentOps typically includes four key stages: monitoring, anomaly detection, root cause analysis, and resolution, together contributing to the accountable, observable agents.

To achieve this, we need open observability standards, attribution methods that work online, benchmarks that reflect real-world conditions, and governance frameworks that keep humans in the loop. Progress has been made on individual fronts, but an integrated, system-level solution remains absent. We believe AgentOps should be a first-class research focus, not a tooling afterthought, if we are to move from brittle prototypes to reliable production systems.

6.4 Expanding the Horizon: Trajectory Analysis in the Wild

Beyond controlled settings, three directions connect trajectory analysis to real-world failure diagnose.

6.4.1 Learning from Production Deployments

Current benchmarks and datasets are predominantly constructed from controlled environments or synthetic error injection. While valuable for establishing baselines, they may not capture the full diversity and unpredictability of failures encountered in real-world deployed agents. Production environments introduce complexities that controlled settings cannot replicate: ambiguous user requests, evolving external APIs, fluctuating latency, and unanticipated edge cases [89]. Moreover, failures in deployment often intertwine with system-level issues (e.g., resource constraints, network instability) that are absent from laboratory evaluations.

This gap presents a potential opportunity for empirical software engineering research, e.g., for large-scale, longitudinal studies that collect trajectory data from production environments, documenting not only failures but also successful recoveries, adaptation patterns, and the long-term effects of system updates. Such empirical foundations would enable more realistic evaluation of attribution and repair techniques, and inspire methods tailored to practical constraints for online diagnosis.

6.4.2 Domain-Specific Failure Diagnose

Different application domains impose unique requirements and introduce distinct failure modes that generic trajectory analysis methods may overlook. Consider coding agents (e.g., SWE-agent [90], AutoCodeRover [91]), which navigate large codebases, generate patches, and interact with version control systems. Their trajectories contain rich structural code snippets, dependency graphs, test outputs which generic natural language methods fail to exploit. Similarly, GUI agents [92] interact with dynamic visual interfaces, interpreting screen layouts, handling UI events, and adapting to application state changes. Their failures often stem from misinterpreting visual elements, incorrect action sequences, or inability to handle unexpected interface transitions.

Other domains, such as web navigation agents and embodied agents in robotics, each exhibit characteristic failure patterns. Developing specialized diagnostic tools that incorporate domain knowledge (e.g., code structure, screen layouts, dialogue acts) could significantly improve the accuracy and actionability of attribution methods. Moreover, cross-domain transfer learning might reveal generalizable principles of agent failure, informing the design of more robust diagnose techniques.

6.4.3 Beyond Task Completion: Constraint Fulfillment, Alignment, and Safety

Most existing failure attribution and enhancement techniques frame success and failure solely in terms of task completion, i.e., whether the agent achieved the stated objective. However, in real-world deployments, an agent may accomplish the task while violating important constraints, compromising safety, or acting against human values. For example, an agent might delete backup data to free disk space, or remove encryption to improve execution speed, thereby satisfying the immediate goal but creating unacceptable risks. Such behaviors are rarely captured by current benchmarks.

Addressing this gap opens several promising research directions. Future benchmarks could extend beyond binary task outcomes to include multi-dimensional annotations. And attribution techniques could be adapted to identify not only where a failure occurs but also what kind of behavioral violation has taken place, and whether the violation was recoverable. By expanding the notion of failure beyond task success, the community can build agents that are not only effective but also trustworthy and aligned with human expectations.

7 RELATED WORK

7.1 LLM Agents for Software Engineering Tasks

The software engineering community has rapidly embraced LLMbased agents to automate a wide spectrum of development activities. Multiagent frameworks such as ChatDev [93] and MetaGPT [94] simulate collaborative software teams that write code, conduct code reviews, and manage projects from highlevel specifications. Specialized agents like SWEagent [90], AutoCodeRover [91] and ExpeRepair [95] tackle realworld GitHub issues by navigating repositories, reproducing bugs, and generating patches, achieving stateoftheart results on benchmarks such as SWEBench. Beyond coding, agents have been deployed for unit test case generation [96], fault localization [97], and automated software testing [98], demonstrating their potential to reshape software engineering workflows.

However, the adoption of these agents introduces new reliability challenges as demonstrated in Section 1. Bouzenia and Pradel [12] conducted the first empirical study of thought-action-result trajectories of software engineering agents, revealing recurrent failure modes such as repeated identical actions without follow-up, and debugging anti-patterns, such as generating a fix without testing it. Their work highlights a pressing need for systematic, automated techniques to diagnose why an agent fails and how to repair its behavior. This survey directly addresses that need.

7.2 Automatic Construction and Self-Evolution of LLM Agents

Parallel to the software engineering communitys focus on deploying agents for development tasks, the artificial intelligence community has intensively explored how to automatically construct, optimize, and evolve LLM agents from scratch or from minimal seeds.

Early work framed agent customization as prompt tuning or incontext learning, but recent advances treat the entire agent its instructions, policies, tool use patterns, and even collaborative workflows as optimizable parameters. For example, SCOPE [50] formulates prompt evolution as an online learning problem, balancing tactical error resolution with strategic principle refinement via a dualstream mechanism and perspectivedriven exploration. Another line of research focuses on optimizing the multiagent collaboration structure through systematic exploration of the design space, driven by diagnostic signals extracted from trajectories.. For example, Maestro [47] jointly searches over graph topologies and agent configurations under token or rollout budgets, guided by reflective textual feedback from traces.

Despite their diversity, all these automatic construction techniques depend critically on understanding why an agent succeeds or fails precisely the focus of this survey.

7.3 AgentOps: Observability and Operational Management for LLM Agents

Parallel to the technical advances in failure attribution and system enhancement surveyed in this paper, a complementary discipline has emerged at the intersection of software engineering, DevOps, and AI infrastructure:

AgentOps. While our work focuses more on how to analyze trajectories to diagnose and repair failures, AgentOps addresses the broader question of how to observe, monitor, and manage agents throughout their entire production lifecycle, such as AgentOps SDK [86], IBM Telemetry with watsonx Orchestrate [87].

Dong et. al. [67] propose the systematic taxonomy of AgentOps, identifying traceable artifacts across the agent production lifecycle and emphasizing observability as a prerequisite for reliability. Moshkovich and Zeltyn [60] articulate a six-stage AgentOps automation pipeline that explicitly integrates root cause analysis and optimization recommendation. Wang et. al. [88] conducts a survey, in which they provide detailed definitions and explanations of the four key stages of AgentOps: monitoring, anomaly detection, root cause analysis, and resolution. We can see that, they all have stages that correspond to the failure attribution and system enhancement dimensions central to our survey.

Existing AgentOps literature are mainly industry white papers, platform documentation, and early-stage academic taxonomies. And they focus on observability infrastructure, platform architecture, and operational best practices rather than the analysis of attribution approaches or enhancement strategies. Our work complements this emerging discipline by offering the first structured roadmap of the diagnostic techniques that AgentOps platforms can (and increasingly do) operationalize.

8 CONCLUSION

Trajectory analysis has rapidly emerged as the central lens through which LLM agents can be understood, diagnosed, and improved. This survey has provided a systematic overview of the emerging field of trajectory analysis for failure attribution and system enhancement in LLM agents. By collecting and synthesizing 55 relevant papers published between early 2025 and April 2026, we have mapped the research landscape along five key dimensions: failure taxonomy, failure attribution, system enhancement and optimization, trajectory visualization and analysis tools, as well as datasets and benchmarks.

Our analysis reveals a field in rapid formation, driven by the recognition that traditional software diagnosis techniques are ill-suited for the non-deterministic, language-heavy, and interaction-intrinsic nature of agent failures. By highlighting both achievements and persistent gaps, we aim to guide researchers and practitioners toward the next generation of robust, accountable, and self-improving agent ecosystems. The journey from today's brittle prototypes to reliable, deployable agentic systems will depend critically on our ability to analyze, attribute, and act upon the rich information embedded in every agent trajectory. We hope this work accelerates that journey.

REFERENCES

- [1] A. Karpathy, "State of GPT," Talk at Microsoft Build, 2023. [Online]. Available: <https://www.youtube.com/watch?v=bZQun8Y4L2A>
- [2] Y. Lou, A. Ghanbari, X. Li, L. Zhang, H. Zhang, D. Hao, and L. Zhang, "Can automated program repair refine fault localization? a unified debugging approach," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 75–87.
- [3] M. Wen, J. Chen, Y. Tian, R. Wu, D. Hao, S. Han, and S.-C. Cheung, "Historical spectrum based fault localization," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2348–2368, 2019.
- [4] N. Islam, R. S. Ayon, D. G. Thomas, S. Ahmed, and M. Wardat, "When agents fail: A comprehensive study of bugs in llm agents with automated labeling," *arXiv preprint arXiv:2601.15232*, 2026.
- [5] N. Nashid, D. Ding, K. Gallaba, A. E. Hassan, and A. Mesbah, "Beyond accuracy: Behavioral dynamics of agentic multi-hunk repair," *arXiv preprint arXiv:2511.11012*, 2025.
- [6] R. Ramakrishnan, "Coding agents don't need to be perfect, they need to recover: Error analysis of 8 models on agentic coding tasks," Snorkel AI Blog, Feb. 2026. [Online]. Available: <https://snorkel.ai/blog/coding-agents-dont-need-to-be-perfect-they-need-to-recover/>
- [7] R. Ramesh, "Ai boss fails spectacularly in month-long business test: Anthropic claude agent loses money, hoards tungsten, believes it's human," AIToday, Jun. 2025. [Online]. Available: <https://www.aitoday.io/ai-boss-fails-spectacularly-in-month-long-business-test-a-28862>
- [8] Real Story Group, "From agentic pilot to production, part 2: Disobedient or just probabilistic?" Real Story Group Blog, Feb. 2026. [Online]. Available: <https://realstorygroup.com/Blog/agentic-pilot-production-part-2-disobedient-or-just-probabilistic>
- [9] T. Dong, H. Sampath, J. Y. Lee, S. Y. Shi, and A. Macvean, "From correctness to collaboration: Toward a human-centered framework for evaluating ai agent behavior in software engineering," *arXiv preprint arXiv:2512.23844*, 2025.
- [10] L. Tang, J. Mei, D. Liu, C. Qian, D. Cheng, J. Shao, and X. Hu, "Interpreting emergent extreme events in multi-agent systems," 2026. [Online]. Available: <https://arxiv.org/abs/2601.20538>
- [11] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen et al., "Which Agent causes task failures and when? on automated failure attribution of LLM Multi-Agent systems," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 1–12, spotlight paper.
- [12] I. Bouzenia and M. Pradel, "Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories," in *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 1–12.
- [13] C. Zhang, J. Yang, D. Yan, S. Yang, and Y. Chen, "Automated breakpoint generation for debugging," *J. Softw.*, vol. 8, no. 3, pp. 603–616, 2013.
- [14] Y. Chen, S. Zhang, Q. Guo, L. Li, R. Wu, and T. Chen, "Deterministic replay: A survey," *ACM Computing Surveys (CSUR)*, vol. 48, no. 2, pp. 1–47, 2015.
- [15] J. Xu, R. Yang, Y. Huo, C. Zhang, and P. He, "Divlog: Log parsing with prompt enhanced in-context learning," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.
- [16] D. Yuan, S. Park, and Y. Zhou, "Characterizing logging practices in open-source software," in *2012 34th international conference on software engineering (ICSE)*. IEEE, 2012, pp. 102–112.
- [17] D. Yuan, J. Zheng, S. Park, Y. Zhou, and S. Savage, "Improving software diagnosability via log enhancement," *ACM Transactions on Computer Systems (TOCS)*, vol. 30, no. 1, pp. 1–28, 2012.
- [18] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 492–504.
- [19] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *2017 IEEE international conference on web services (ICWS)*. IEEE, 2017, pp. 33–40.
- [20] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "An evaluation study on log parsing and its use in log mining," in *2016 46th annual IEEE/IFIP international conference on dependable systems and networks (DSN)*. IEEE, 2016, pp. 654–661.
- [21] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 707–740, 2016.
- [22] R. Lu, Y. Li, and Y. Huo, "Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks," in *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 1–12, new Ideas Track.
- [23] D. Guo, J. Wu, and S. M. Yiu, "Agenteval: Dag-structured step-level evaluation for agentic workflows with error propagation tracking," *arXiv preprint arXiv:2604.23581*, 2026.

- [24] K. Zhu, Z. Liu, B. Li, M. Tian, Y. Yang, J. Zhang, P. Han, Q. Xie, F. Cui, W. Zhang, X. Ma, X. Yu, G. Ramesh, J. Wu, Z. Liu, P. Lu, J. Zou, and J. You, "Where LLM Agents Fail and How They can Learn From Failures," 2025. [Online]. Available: <https://arxiv.org/abs/2509.25370>
- [25] D. Deshpande, V. Gangel, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian, "TRAIL: Trace Reasoning and Agentic Issue Localization," 2025. [Online]. Available: <https://arxiv.org/abs/2505.08638>
- [26] S. Barke, A. Goyal, A. Khare, A. Singh, S. Nath, and C. Bansal, "Agentrx: Diagnosing ai agent failures from execution trajectories," 2026. [Online]. Available: <https://arxiv.org/abs/2602.02475>
- [27] X. Ma, X. Xie, Y. Wang, J. Wang, B. Wu, M. Li, and Q. Wang, "Demystifying the lifecycle of failures in platform-orchestrated agentic workflows," 2026.
- [28] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why Do Multi-Agent LLM Systems Fail?" in *Proceedings of the 2nd Workshop on Building Trust in Machine Learning at ICLR 2025*, 2025, pp. 1–15.
- [29] K. Song, A. Jayarajan, Y. Ding, Q. Su, Z. Zhu, S. Liu, and G. Pekhimenko, "Aegis: Taxonomy and Optimizations for Overcoming Agent-Environment Failures in LLM Agents," 2025. [Online]. Available: <https://arxiv.org/abs/2508.19504>
- [30] Y. Ge, L. Xie, Z. Li, Y. Pei, and T. Zhang, "Who is Introducing the Failure? Automatically Attributing Failures of Multi-Agent Systems via Spectrum Analysis," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2026, pp. 1–12.
- [31] A. Barrak, "Traceability and Accountability in Role-Specialized Multi-Agent LLM Pipelines," 2025. [Online]. Available: <https://arxiv.org/abs/2510.07614>
- [32] Y. Yu, M. Li, S. Xu, J. Fu, X. Hou, F. Lai, and B. Wang, "CORRECT: Condensed error Recognition via Knowledge Transfer in Multi-Agent Systems," 2025. [Online]. Available: <https://arxiv.org/abs/2509.24088>
- [33] K. Sun, W. Li, B. Dong, Y. Lin, J. Zhang, and B. Shi, "Scope Delineation Before Localization: A Two-Stage Framework for Enhancing Failure Attribution in Multi-Agent Systems," in *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI)*, 2026, pp. 1–10.
- [34] X. Zhao, S. Liu, Y. Zhang, Q. Ma, G. Cheng, N. Wang, and C. Liu, "Promas: Proactive error forecasting for multi-agent systems using markov transition dynamics," *arXiv preprint arXiv:2603.20260*, 2026.
- [35] A. Banerjee, A. Nair, and T. Borogovac, "Where Did It All Go Wrong? A Hierarchical Look into Multi-Agent Error Attribution," in *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS)*, 2025, pp. 1–15.
- [36] C. Zhu, S. Hong, J. Wu, K. Chawla, C. Tang, Y. Yin, N. Wolfe, E. Babinsky, and D. Liu, "RAFFLES: Reasoning-based Attribution of Faults for LLM Systems," in *Proceedings of the 1st Workshop on Multi-Task and In-Context Learning with Large Language Models (MTI-LLM) at NeurIPS 2025*, 2025, pp. 1–12.
- [37] G. Ma, J. Zhu, H. Guo, W. Shi, J. Shen, J. Liu, and Y. Liang, "Automatic Failure Attribution and Critical Step Prediction Method for Multi-Agent Systems Based on Causal Inference," 2025. [Online]. Available: <https://arxiv.org/abs/2509.08682>
- [38] A. West, Y. Weng, M. Zhu, Z. Lin, Z. Ning, and Y. Zhang, "Abduct, Act, Predict: Scaffolding Causal Inference for Automated Failure Attribution in Multi-Agent Systems," in *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS)*, 2025, pp. 1–15.
- [39] Y. Wang, W. Wu, J. Wang, and Q. Wang, "From flat logs to causal graphs: Hierarchical failure attribution for llm-based multi-agent systems," 2026.
- [40] H. Li, Y. Yao, L. Zhu, R. Feng, H. Ye, J. Wang, Y. He, P. Zou, L. Zhang, X. Lei et al., "Codetracer: Towards traceable agent states," *arXiv preprint arXiv:2604.11641*, 2026.
- [41] J. Li, E. Yilmaz, B. Chen, and D.-T. Le, "Towards self-improving error diagnosis in multi-agent systems," *arXiv preprint arXiv:2604.17658*, 2026.
- [42] G. Zhang, J. Wang, J. Chen, W. Zhou, K. Wang, and S. Yan, "AgenTracer: Who Is Inducing Failure in the LLM Agentic Systems?" in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2026, pp. 1–18.
- [43] H. Zhang, Y. Shi, X. Gu, H. You, Z. Zhang, L. Gan, Y. Yuan, and J. Huang, "Graphtracer: Graph-guided failure tracing in llm agents for robust multi-turn deep search," 2025. [Online]. Available: <https://arxiv.org/abs/2510.10581>
- [44] F. Kong, R. Zhang, H. Yin, G. Zhang, X. Zhang, Z. Chen, Z. Zhang, X. Zhang, S.-C. Zhu, and X. Feng, "Aegis: Automated Error Generation and Attribution for Multi-Agent Systems," 2025. [Online]. Available: <https://arxiv.org/abs/2509.14295>
- [45] M. Ma, J. Zhang, F. Yang, Y. Kang, Q. Lin, T. Yang, S. Rajmohan, and D. Zhang, "DoVer: Intervention-Driven Auto Debugging for LLM Multi-Agent Systems," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2026, pp. 1–16.
- [46] M. Chen, J. Wang, F. Mu, Y. Wang, Z. Liu, H. Feng, and Q. Wang, "Traceelephant: Seeing the whole elephant for failure attribution in llm-based multi-agent systems," in *Proceedings of ACL 2026*, 2026, <https://github.com/TraceElephant/TraceElephant>.
- [47] W. Wang, P. Kattakinda, and S. Feizi, "Maestro: Joint Graph & Config Optimization for Reliable AI Agents," 2025. [Online]. Available: <https://arxiv.org/abs/2509.04642>
- [48] J. Zhang, K. Cai, Q. Zeng, N. Liu, S. Fan, Z. Chen, and K. Wang, "Failure-Driven Workflow Refinement," 2025. [Online]. Available: <https://arxiv.org/abs/2510.10035>
- [49] R. Costa, "Instruction-Level Weight Shaping: A Framework for Self-Improving AI Agents," 2025. [Online]. Available: <https://arxiv.org/abs/2509.00251>
- [50] Z. Pei, H.-L. Zhen, S. Kai, S. J. Pan, Y. Wang, M. Yuan, and B. Yu, "SCOPE: Prompt Evolution for Enhancing Agent Effectiveness," 2025. [Online]. Available: <https://arxiv.org/abs/2512.15374>
- [51] D. Zhang, "AgentDevel: Reframing Self-Evolving LLM Agents as Release Engineering," 2026. [Online]. Available: <https://arxiv.org/abs/2601.04620>
- [52] Z. Hao, H. Wang, J. Luo, J. Zhang, Y. Zhou, Q. Lin, C. Wang, H. Dong, and J. Chen, "Recreate: Reasoning and creating domain agents driven by experience," 2026. [Online]. Available: <https://arxiv.org/abs/2601.11100>
- [53] K. Zhang, H. Zhang, G. Li, J. You, J. Li, Y. Zhao, and Z. Jin, "SEAlign: Alignment training for software engineering agent," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE 2026)*. ACM/IEEE, 2026.
- [54] S. Liu, Y. Chen, R. Krishna, S. Sinha, J. Ganhotra, and R. Jabbarvand, "Process-centric analysis of agentic software systems," *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA1, pp. 1961–1988, 2026.
- [55] J. Ni, Y. Liu, X. Liu, Y. Sun, M. Zhou, P. Cheng, D. Wang, E. Zhao, X. Jiang, and G. Jiang, "Trace2skill: Distill trajectory-local lessons into transferable agent skills," *arXiv preprint arXiv:2603.25158*, 2026.
- [56] Y.-A. Xiao, P. Gao, C. Peng, and Y. Xiong, "Improving the Efficiency of LLM Agent Systems through Trajectory Reduction," 2025. [Online]. Available: <https://arxiv.org/abs/2509.23586>
- [57] F. Lin, S. Chen, R. Fang, H. Wang, and T. Lin, "Stop Wasting Your Tokens: Towards Efficient Runtime Multi-Agent Systems," 2025. [Online]. Available: <https://arxiv.org/abs/2510.26585>
- [58] R. Nanda, C. Maddila, S. Jha, E. M. Khan, M. Paltenghi, and S. Chandra, "Wink: Recovering from misbehaviors in coding agents," *arXiv preprint arXiv:2602.17037*, 2026.
- [59] Y. Zheng, Y. Hu, T. Yu, and A. Quinn, "AgentSight: System-Level Observability for AI Agents Using eBPF," in *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*, ser. SOSP '25. ACM, Oct. 2025, pp. 110–115. [Online]. Available: <http://dx.doi.org/10.1145/3766882.3767169>
- [60] D. Moshkovich and S. Zeltyn, "Taming uncertainty via automation: Observing, analyzing, and optimizing agentic ai systems," 2025. [Online]. Available: <https://arxiv.org/abs/2507.11277>
- [61] T. Ou, W. Guo, A. Gandhi, G. Neubig, and X. Yue, "AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, ser. EMNLP'25, 2025, pp. 1–8.
- [62] M. Desmond, J. Y. Lee, I. Ibrahim, J. M. Johnson, A. Sil, J. MacNair, and R. Puri, "Agent Trajectory Explorer: Visualizing and Providing Feedback on Agent Trajectories," in *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, ser. AAAI'25/IAAI'25/EAAI'25. AAAI Press, 2025. [Online]. Available: <https://doi.org/10.1609/aaai.v39i28.35350>

- [63] W. Epperson, G. Bansal, V. C. Dibia, A. Fourney, J. Gerrits, E. E. Zhu, and S. Amershi, "Interactive Debugging and Steering of Multi-Agent AI Systems," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI '25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: <https://doi.org/10.1145/3706598.3713581>
- [64] X. Wang, M. Yin, E. Koh, and M. D. Dogan, "Xagen: An Explainability Tool for Identifying and Correcting Failures in Multi-Agent Workflows," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–12.
- [65] R. Sheng, Y. Yang, C. Shi, Y. Lin, Z. Chen, H. Qu, and F. Cheng, "Dills: Interactive diagnosis of llm-based multi-agent systems via layered summary of agent behaviors," in *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, ser. CHI '26. New York, NY, USA: Association for Computing Machinery, 2026.
- [66] Y. In, M. Tanjim, J. Subramanian, S. Kim, U. Bhattacharya, W. Kim, S. Park, S. Sarkhel, and C. Park, "Rethinking failure attribution in multi-agent systems: A multi-perspective benchmark and evaluation," *arXiv preprint arXiv:2603.25001*, 2026.
- [67] L. Dong, Q. Lu, and L. Zhu, "Agentops: Enabling observability of llm agents," 2024. [Online]. Available: <https://arxiv.org/abs/2411.05285>
- [68] J. Roig, "How Do LLMs Fail In Agentic Scenarios? A Qualitative Analysis of Success and Failure Scenarios of Various LLMs in Agentic Simulations," in *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS)*, 2025, pp. 1–15.
- [69] G. Fang, V. Isahagian, K. Jayaram, R. Kumar, V. Muthusamy, P. Oum, and G. Thomas, "Trajectory-informed memory generation for self-improving agent systems," *arXiv preprint arXiv:2603.10600*, 2026.
- [70] T. Ma, Y. Chen, V. Anand, A. Cornacchia, A. R. Faustino, G. Liu, S. Zhang, H. Luo, S. A. Fahmy, Z. A. Qazi, and M. Canini, "MAESTRO: Multi-Agent Evaluation Suite for Testing, Reliability, and Observability," 2026. [Online]. Available: <https://arxiv.org/abs/2601.00481>
- [71] L. Advani, "Trajectory guard a lightweight, sequence-aware model for real-time anomaly detection in agentic AI," in *AAAI 2026 Workshop on Trust and Control in Agentic AI (TrustAgent)*, 2026, to appear.
- [72] S. Chaduvula, J. Ho, K. Kim, A. Narayanan, M. Alinoori, M. Garg, D. Ramachandram, and S. Raza, "From features to actions: Explainability in traditional and agentic ai systems," 2026. [Online]. Available: <https://arxiv.org/abs/2602.06841>
- [73] S. Chen, A. Hafeez, and S. Paracha, "Signals: Trajectory sampling and triage for agentic interactions," *arXiv preprint arXiv:2604.00356*, 2026.
- [74] Z. Chen, W. Ma, and L. Jiang, "Beyond final code: A process-oriented error analysis of software development agents in real-world github scenarios," *arXiv preprint arXiv:2503.12374*, 2025.
- [75] T. Mehtiyev and W. Assunção, "Beyond resolution rates: Behavioral drivers of coding agent success and failure," *arXiv preprint arXiv:2604.02547*, 2026.
- [76] C. Han, X. Liu, H. Wang, S. Li, J. Yang, H. Jiang, Z. Wang, Q. Yin, L. Qiu, C. Yu, Y. Gao, Z. Li, B. Yin, J. Shang, and H. Ji, "Can language models follow multiple turns of entangled instructions?" in *Findings of the Association for Computational Linguistics: EMNLP 2025*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 25445–25460. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.1387/>
- [77] Z. Zeng, J. Yu, T. Gao, Y. Meng, T. Goyal, and D. Chen, "Evaluating large language models at evaluating instruction following," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=tr0KidwPLc>
- [78] J. A. Jones, M. J. Harrold, and J. Stasko, "Visualization of test information to assist fault localization," in *Proceedings of the 24th international conference on Software engineering*, 2002, pp. 467–477.
- [79] M. Monperrus, "Automatic software repair: A bibliography," *ACM Computing Surveys (CSUR)*, vol. 51, no. 1, pp. 1–24, 2018.
- [80] Y. Zhu, J. Liu, M. Guo, Y. Bao, W. Ma, Z. Liu, K. Song, and Y. Yang, "Bestconfig: tapping the performance potential of systems via automatic configuration tuning," in *Proceedings of the 2017 symposium on cloud computing*, 2017, pp. 338–350.
- [81] R. Chillarege, I. S. Bhandari, J. K. Chaar, M. J. Halliday, D. S. Moebs, B. K. Ray, and M.-Y. Wong, "Orthogonal defect classification—a concept for in-process measurements," *IEEE Transactions on software Engineering*, vol. 18, no. 11, pp. 943–956, 1992.
- [82] H. Agrawal, R. A. DeMillo, and E. H. Spafford, "Debugging with dynamic slicing and backtracking," *Software: Practice and Experience*, vol. 23, no. 6, pp. 589–616, 1993.
- [83] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspán, and C. Shanbhag, "Dapper, a large-scale distributed systems tracing infrastructure," 2010.
- [84] F. Elberzhager, J. Münch, and V. T. N. Nha, "A systematic mapping study on the combination of static and dynamic quality assurance techniques," *Information and software technology*, vol. 54, no. 1, pp. 1–15, 2012.
- [85] R. E. Fairley, "Tutorial: Static analysis and dynamic testing of computer software," *Computer*, vol. 11, no. 4, pp. 14–23, 2006.
- [86] AgentOps, Inc., "AgentOps TypeScript SDK," Node Package Manager (NPM), Jun. 2025, version 0.1.0. [Online]. Available: <https://www.npmjs.com/package/agentops>
- [87] IBM Cloud Education, "Agentops: Monitor and govern ai agents with IBM telemetry by using watsonx orchestrate," IBM Think Tutorial, 6 2025, accessed: 2026-02-13. [Online]. Available: <https://www.ibm.com/think/tutorials/agentops-telemetry-watsonx-orchestrate>
- [88] Z. Wang, J. Li, Q. Zhou, H. Si, Y. Liu, J. Li, G. Xie, F. Sun, D. Pei, and C. Pei, "A survey on agentops: Categorization, challenges, and future directions," *CoRR*, vol. abs/2508.02121, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2508.02121>
- [89] Fiddler AI Team, "The production-ready agent: A practical playbook," Fiddler AI Blog, Aug. 2025, covers real-world deployment challenges including architectural trade-offs, silent failures, and the need for continuous observability in production environments. [Online]. Available: <https://www.fiddler.ai/blog/production-ready-agent-playbook>
- [90] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds., 2024. [Online]. Available: http://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
- [91] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, M. Christakis and M. Pradel, Eds. ACM, 2024, pp. 1592–1604. [Online]. Available: <https://doi.org/10.1145/3650212.3680384>
- [92] C. Zhang, S. He, J. Qian, B. Li, L. Li, S. Qin, Y. Kang, M. Ma, G. Liu, Q. Lin *et al.*, "Large language model-brained gui agents: A survey," *arXiv preprint arXiv:2411.18279*, 2024.
- [93] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, to appear. [Online]. Available: <https://arxiv.org/abs/2307.07924>
- [94] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for A multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=VtmBAGCN7o>
- [95] F. Mu, J. Wang, L. Shi, S. Wang, S. Li, and Q. Wang, "EXPERPAIR: dual-memory enhanced llm-based repository-level program repair," *CoRR*, vol. abs/2506.10484, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2506.10484>
- [96] Z. Yuan, M. Liu, S. Ding, K. Wang, Y. Chen, X. Peng, and Y. Lou, "Evaluating and improving chatgpt for unit test generation," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1703–1726, 2024.
- [97] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring llm-based general bug reproduction," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023,

- pp. 2312–2323. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00194>
- [98] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, “Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 2024, pp. 100:1–100:13. [Online]. Available: <https://doi.org/10.1145/3597503.3639180>